

OXYPC INVENTORY LIFECYCLE

ARCHITECTURE

Technical Architecture

System design, data model, integrations, and the decisions behind them.

Last updated: 6 Nov 2026 · Source: staging branch @ 55ce58a

App Development Team · Bluemonk

Technical Architecture

Last updated: 6 Nov 2026 · Source: staging branch @ 55ce58a

In this document: Part A — Solution Architecture · Part B — Data Model & Dictionary · Part C — Integrations · Part D — Architecture Decision Records

PART A

Solution Architecture

This document describes how the OxyPC Inventory Lifecycle app is built: who and what talks to it, what is inside it, how a request flows through it, and the patterns every new feature should follow. Every claim cites a file or commit. Anything not proven by the code is listed under **Open questions** at the end.

---

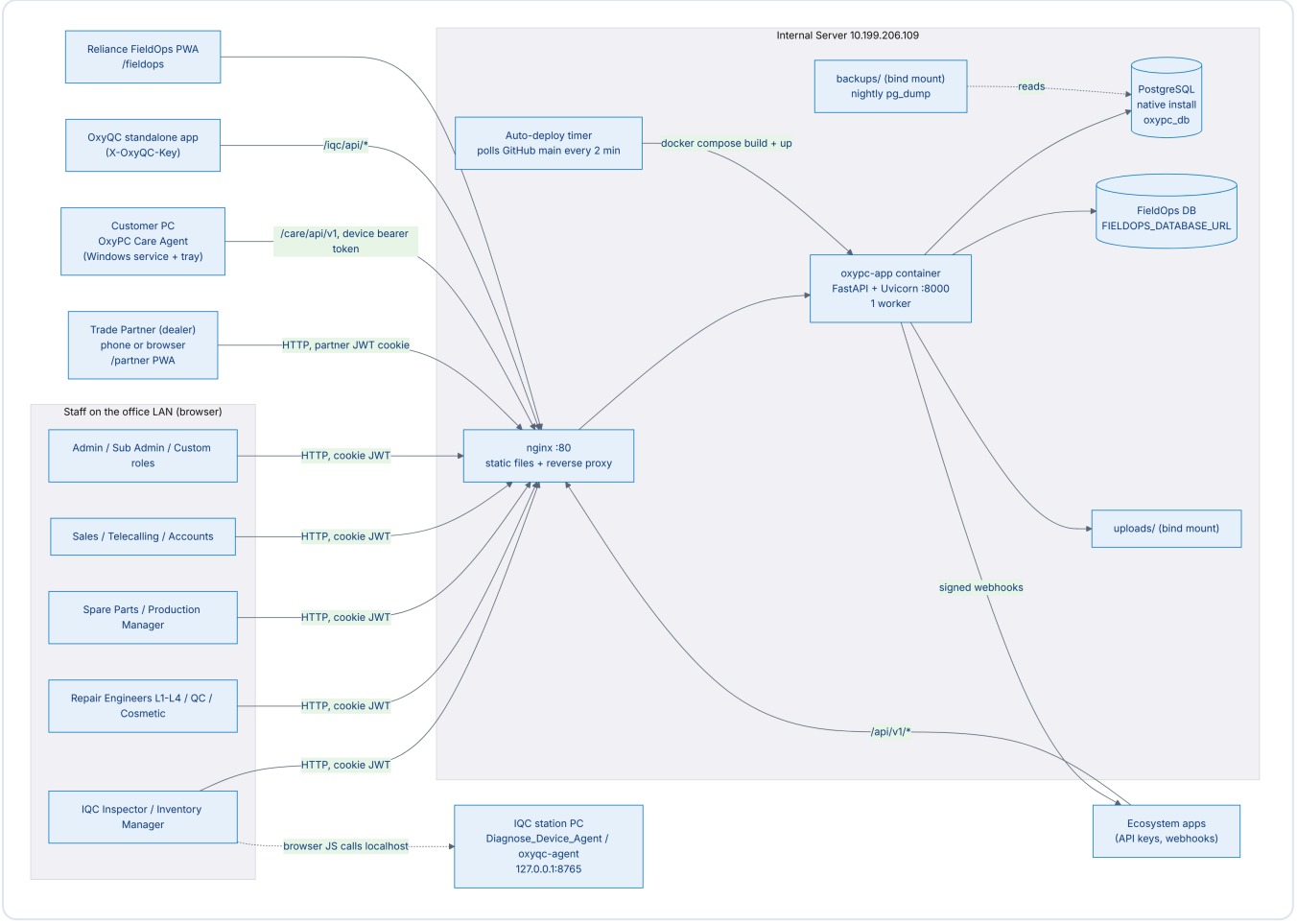
1. Summary

Item	Value	Source
App name	OxyPC Inventory Lifecycle	`config.py` (`APP_NAME`), commit `e5538cf` (rebrand)
Style	Server-rendered web app (FastAPI + Jinja2), one Python process	`main.py` , `templates_config.py`
Database	PostgreSQL, async access through SQLAlchemy 2 + asyncpg	`database.py` , `requirements.txt`
Production	Linux "Internal Server" at `http://10.199.206.109` , app in a Docker container behind nginx	`deploy/nginx-oxypc-internal.conf` , `docs/superpowers/specs/2026-10-01-docker-containerization-design.md` §9
Staging	`http://localhost:8001` on staging machines	`qa_docs/design_system/design_system.html` (Environments row, release table)
Users	30+ concurrent LAN users	`CLAUDE.md`
Size of code base	81 router modules + 9 API v1 modules, 57 model modules, 27 services, 242 Jinja templates, 172 test files	file counts in `routers/` , `routers/api_v1/` , `models/` , `services/` , `templates/` , `tests/`

---

2. System context

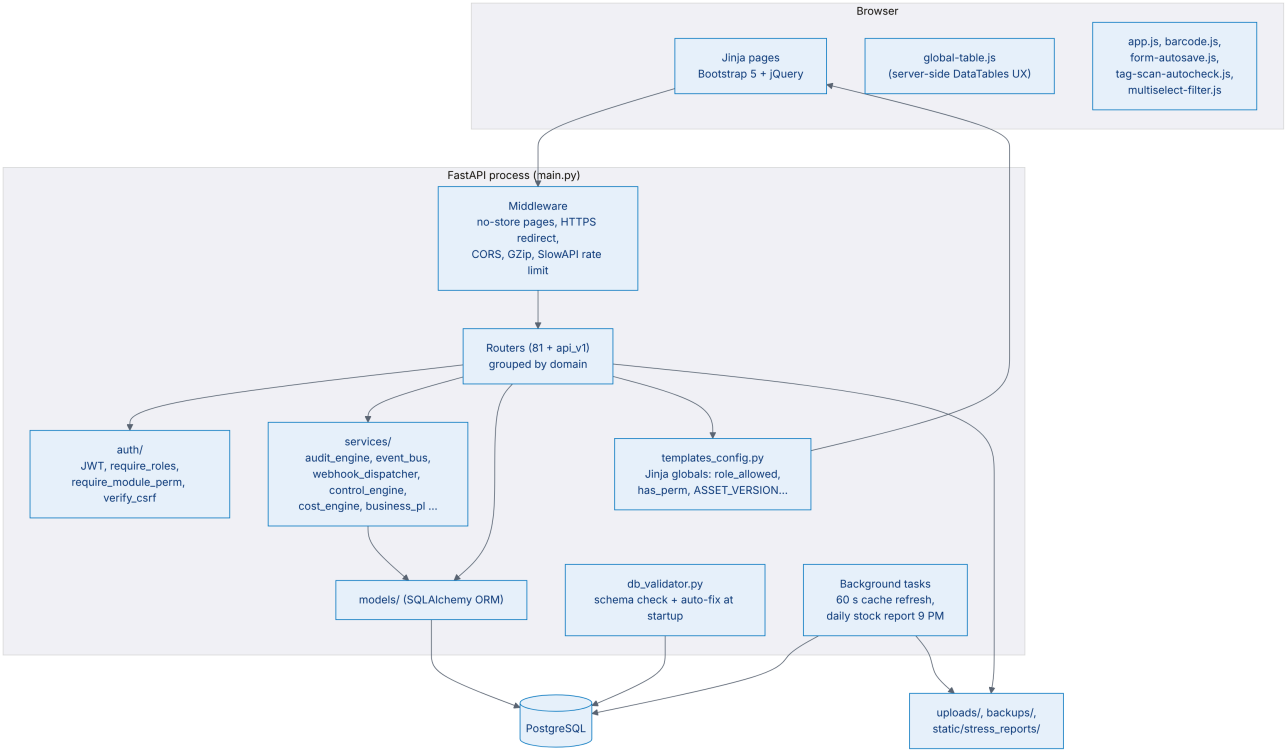
The diagram shows the people and systems around the app. Production nginx serves `/static/` straight from disk and passes everything else to the app (`deploy/nginx-oxypc-internal.conf`).



Actor / system	How it connects	Auth	Source
Staff (all roles)	Browser → nginx → app	`access_token` JWT cookie + CSRF double-submit cookie	`auth/dependencies.py` (`get_current_user` , `verify_csrf`)
Trade Partner (dealer)	`/partner` routes, installable PWA	Separate partner JWT cookie; staff sessions cannot open these routes	`routers/partner_portal.py` header, `auth/partner_auth.py` , `static/partner/manifest.json` , `static/partner/sw.js`
IQC station agent (`oxyqc-agent`)	The IQC page's browser JS calls `http://127.0.0.1:8765/diagnose` on the same PC; the agent is never on the LAN	None (loopback only)	`oxyqc-agent/README_AGENT.txt`
OxyQC standalone app	Machine API `/iqc/api/*`	Static key in `X-OxyQC-Key` header	`routers/iqc_api.py` header, `config.py` (`OXYQC_API_KEY`)
Customer Care Agent	`/care/api/v1`	Device-bound bearer token, scoped to one pairing	`routers/care_api.py` header, `auth/care_auth.py` , `care_agent_windows/README.md`
Ecosystem apps	`/api/v1/*` JSON API; outbound HMAC-signed webhooks	API keys with scopes; CORS allow-list	`routers/api_v1/` , `auth/api_key.py` , `services/webhook_dispatcher.py` , `config.py` (`ALLOWED_ORIGINS`)
Reliance FieldOps	`/fieldops` , offline-first PWA	Own users and own database	`routers/fieldops.py` header, `fieldops_db.py` , `README.md`

Actor / system	How it connects	Auth	Source
Telecalling mobile	Mobile PWA with offline queue	Staff login	`static/m/telecalling/` (`manifest.json` , `sw.js` , `offline-queue.js`)
WhatsApp service	**Disabled.** Router not registered	n/a	`main.py` lines 266-267

3. Container view



3.1 Routers grouped by domain

The app has one router file per feature area (`CLAUDE.md` "Architecture"). All are registered in `main.py` lines 234-316.

Domain	Router files (in `routers/`)
Inbound and IQC	`grn.py` , `iqc.py` , `iqc_api.py` , `bulk_upload.py` , `stock.py` , `parts_grn.py` , `physical_audits.py` , `inventory_location.py` , `block_tags.py` , `entity_movement.py` , `transfers.py`
Production, repair and QC	`devices.py` , `repair.py` , `qc.py` , `cosmetic.py` , `buckets.py` , `stage_control.py` , `workid_status.py` , `part_requests.py` , `part_estimation.py` , `spare_parts.py` , `procure_dashboard.py` , `product_master.py` , `scrap.py` , `scrap_for_sale.py` , `stress_api.py`
Sales and dispatch	`sales.py` , `dispatch.py` , `gate_pass.py` , `invoices.py` , `quotations_list.py` , `dealer_quotations.py` , `part_sales.py` , `extended_warranty.py` , `market.py` , `terms_conditions.py`

Domain	Router files (in `routers/`)
CRM, dealers, telecalling	`crm_dashboard.py`, `crm_contacts.py`, `crm_sourcing.py`, `crm_sales.py`, `crm_quotes.py`, `crm_activities.py`, `crm_price_matrix.py`, `crm_purchase_orders.py`, `crm_reports.py`, `crm_assign_leads.py`, `deals.py`, `accounts.py`, `dealers.py`, `assign_dealer_leads.py`, `telecalling.py`, `m_telecalling.py`, `telesales_dashboard.py`, `model_requests.py`, `model_requested.py`
Finance and reports	`partner_payments.py`, `reports.py`, `dashboard.py`
External parties	`partner_portal.py`, `partner_admin.py`, `care_api.py`, `care_internal.py`, `care_admin.py`, `fieldops.py`, `api.py`, `api_v1/*` (api_keys, devices, health, iqc, lots, sales, spare_parts, telecalling, webhooks)
Platform and admin	`auth.py`, `admin.py`, `master.py`, `settings.py`, `company_settings.py`, `sidebar_config.py`, `landing_pages.py`, `trash.py`, `notifications.py`, `manuals.py`, `health.py`, `qa_uat.py`, `attendance.py`, `attendance_group_config.py`, `tickets.py`
Disabled	`whatsapp.py` (import and registration commented out, `main.py` lines 176, 266-267)

Note: `routers/repair.py.bak` is a stray backup file in the router folder.

3.2 Services

Service	Job	Source
`audit_engine.audit()`	Writes an `AuditLog` row for a significant action. Does not commit; the caller commits. Captures user, action, table, record id, old/new values, IP	`services/audit_engine.py`
`event_bus`	In-process pub/sub. Routes call `publish()` after commit; handlers run as FastAPI background tasks after the response	`services/event_bus.py`
`webhook_dispatcher`	Subscribed at startup to 7 event types; sends HMAC-signed webhooks	`services/webhook_dispatcher.py`, `main.py` startup
`control_engine`	Stage-transition rules (cached)	`services/control_engine.py`, `main.py` (`invalidate_transitions_cache`)
`daily_stock_reports`	Generates Daily Stock exports every day at 9 PM	`services/daily_stock_reports.py`, `main.py` startup
Others	Costing, P&L, pricing, part estimation, warranty/care, GRN stocking, invoice parsing, PO PDF	`services/*.py`

3.3 Background jobs (inside the app process)

There is no separate scheduler or worker. Jobs are `asyncio` tasks started in `startup_event()` (`main.py`).

Job	Interval	Purpose	Source
Cache refresh	Every 60 s	Reloads permissions, sidebar labels, page titles, breadcrumbs, master dropdowns, module visibility; clears transition cache	`main.py` `_refresh_all_caches_periodically`
Daily stock report	Checks every few minutes; generates at 9 PM	Writes Daily Stock export files	`services/daily_stock_reports.py` `scheduler_loop`
Event handlers	Per request	Webhook delivery after response	`services/event_bus.py`

Job	Interval	Purpose	Source
One-time backfills	Each startup (idempotent)	GRN-Lot links, GRN entity, tag backfills, size-unit spacing, Product Master seeding, QA release tracking	`main.py` startup blocks

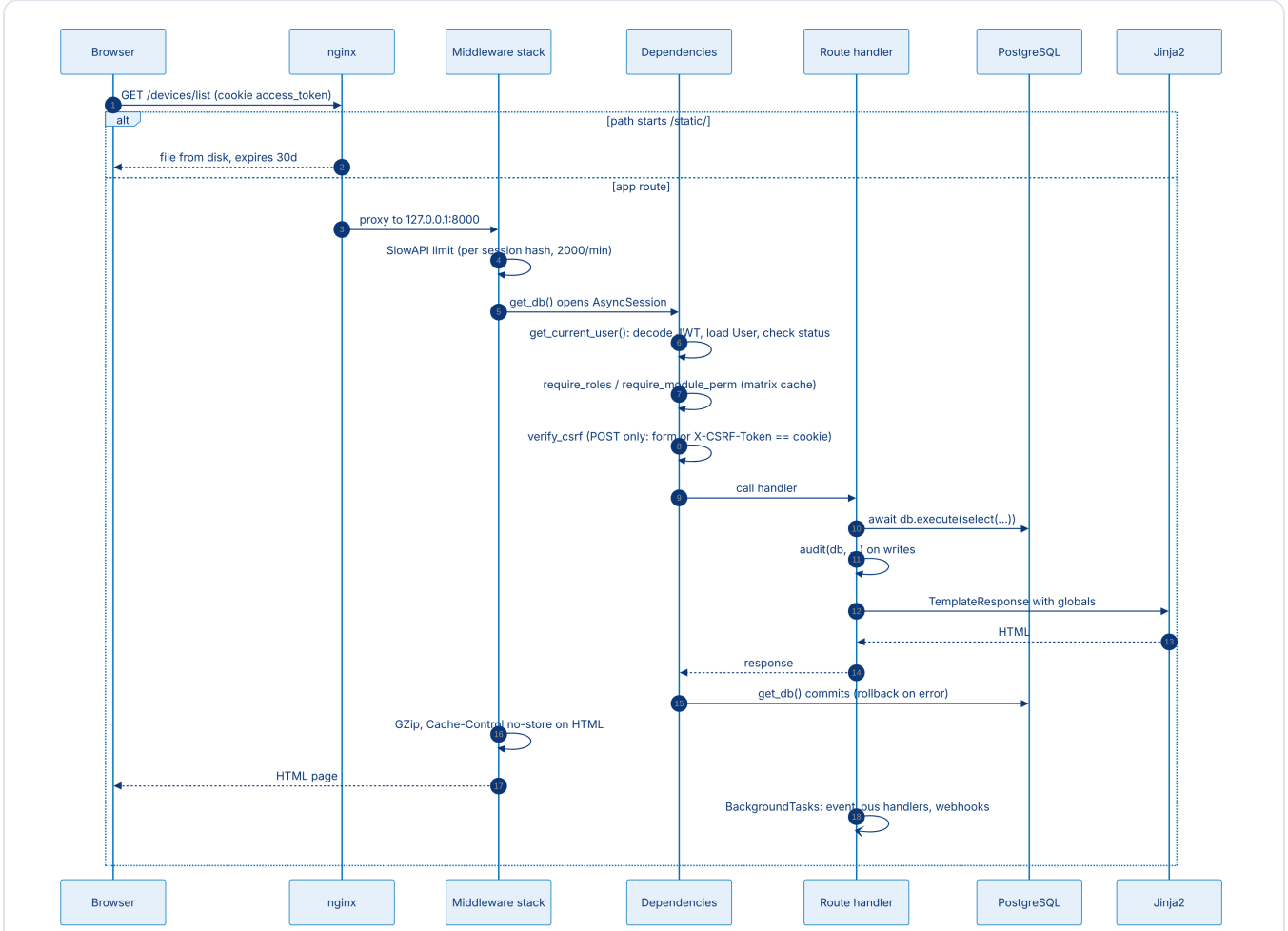
Host-level jobs (outside the app) are covered in 06 Deployment: nightly backup timer and auto-deploy timer.

3.4 Front end

Part	Detail	Source
Templates	242 Jinja2 files; all pages extend `base.html`	`templates/`, `qa_docs/design_system/design_system.html` checklist
Jinja globals	`role_allowed`, `has_perm`, `has_explicit_perm`, `any_perm`, `can_view_pricing`, `can_set_price`, `module_hidden`, `sidebar_label`, `master_options`, `ASSET_VERSION` and more	`templates_config.py` lines 43-330
Global Table	One JS module that layers the standard heavy-table UX on a server-side DataTables instance (sticky first column, one-line headers, 12 rows default, cell clamping, scan-to-select)	`static/js/global-table.js` header, commit `ae43e3f`
Cache busting	`?v={{ ASSET_VERSION }}` from asset mtimes	`templates_config.py` `_VERSIONED_ASSETS`, `tests/test_asset_version_cache_busting.py`
App zoom	Whole UI at 90 % (`--app-zoom: 0.9`); use `var(--full-vh)` not `100vh`	`static/css/app.css` lines 10-20
Vendor libraries	Served locally, no CDN	`static/vendor/`, design system checklist "No CDN links"

4. Request lifecycle

A normal staff page request (for example opening a list page, then submitting a form):



Step	What happens	Source
Rate limit	Bucket is the hashed session token, or client IP before login. Default 2000/min. Login is 20/min per IP	`limiter.py`, `routers/auth.py` line 55
HTTPS redirect	Only acts if `X-Forwarded-Proto` is present. The LAN nginx deliberately omits it	`main.py` `_force_https`, `deploy/nginx-oxypc-internal.conf`
Auth	JWT in `access_token` cookie, HS algorithm with `OXYPC_SECRET_KEY`; missing/invalid → 302 to `/auth/login`	`auth/dependencies.py` lines 86-109
Authorisation	Built-in roles pass if listed; custom roles pass non-admin gates; otherwise the Permission Matrix can grant the path	`auth/dependencies.py` `require_roles`, `_matrix_grants_path`
CSRF	Double-submit cookie on every non-GET; missing cookie → redirect to login (treated as expiry)	`auth/dependencies.py` `verify_csrf`
Transaction	One session per request; commit after handler, rollback on exception	`database.py` `get_db`
Errors	JSON for API/AJAX callers, HTML modal page otherwise; 500s written to `errors.log`	`main.py` `_wants_json`, `_render_error`, `generic_exception_handler`
Caching	HTML gets `Cache-Control: no-store`; static files are cached and cache-busted	`main.py` `_no_store_pages`

5. Tech stack

Versions come from `requirements.txt` (pinned with `==` unless noted) and from file headers in `static/vendor/`.

Layer	Technology	Version	Source
Language	Python	3.12 in the Docker image; 3.10.12 on the server host; Windows scripts point to 3.13/3.14; Windows bundle 3.11.9	`Dockerfile`, docker spec §9, `run_server.ps1`, `install_service.bat`, `prepare_bundle.bat`
Web framework	FastAPI	0.115.6	`requirements.txt`
ASGI server	uvicorn[standard]	0.32.1	`requirements.txt`
ORM	SQLAlchemy[asyncio]	2.0.36	`requirements.txt`
DB driver	asyncpg	0.30.0	`requirements.txt`
Migrations	Alembic	1.14.0 (36 revisions)	`requirements.txt`, `alembic/versions/`
Database	PostgreSQL	17 on the Internal Server; 16.6 in the Windows bundle	`Dockerfile` comment (pg_dump 17), `prepare_bundle.bat`
Templates	Jinja2	3.1.4	`requirements.txt`
Auth	python-jose[cryptography] 3.3.0, bcrypt 5.0.0	pinned	`requirements.txt` (with bcrypt/passlib note)
Rate limit	slowapi	0.1.9	`requirements.txt`
Config	python-dotenv	1.0.1	`requirements.txt`
Files/exports	openpyxl ≥3.1.0, fpdf2 ≥2.7.9, reportlab ≥4.0.0, python-barcode ≥0.15.1, python-pptx ≥1.0.0, pypdf, pdfplumber (unpinned)	floor or unpinned	`requirements.txt`
Tests	pytest 8.3.4, pytest-asyncio 0.24.0, httpx 0.28.1	pinned	`requirements.txt`, `pytest.ini`
CSS/JS	Bootstrap	5.3.2	`static/vendor/bootstrap/css/bootstrap.min.css` header
Icons	Bootstrap Icons	1.11.3	`static/vendor/bootstrap-icons/bootstrap-icons.css` header
Tables	DataTables	1.13.7	`static/vendor/datatables/js/jquery.dataTables.min.js` header
DOM	jQuery	3.7.1	`static/vendor/jquery/jquery-3.7.1.min.js`
Charts	Chart.js	4.4.0	`static/vendor/chartjs/chart.umd.min.js` header
Select boxes	Tom Select	2.3.1	`static/vendor/tom-select/js/tom-select.complete.min.js` header

Layer	Technology	Version	Source
Container	Docker Engine 29.8.0, Compose v5.5.1	as found on server	docker spec §9
Reverse proxy	nginx	not recorded	`deploy/nginx-oxypc-internal.conf`
Packaging (legacy)	PyInstaller ≥6.10.0, Inno Setup 6		`requirements.txt`, `build_all.bat`, `oxypc_setup.iss`

6. Key design patterns

Each pattern below is a rule for new code. The project skills in `.claude/skills/` enforce several of them.

#	Pattern	Rule	Where it lives
P1	Async SQLAlchemy	`await db.execute(select(...))` then `.scalar_one_or_none()` or `.scalars().all()`. One session per request from `get_db`	`CLAUDE.md`, `database.py`
P2	Soft delete	Rows are flagged `is_trashed` instead of deleted; lists filter them out; a Trash page restores them. On devices, lots, spare parts, parts GRN, scrap, CRM tables the column is now `NOT NULL DEFAULT false`	`models/device.py` l.248, `models/lot.py` l.47, `models/spare_parts.py` l.48, `routers/trash.py`, migrations `20260503_0900_soft_delete_devices.py`, `20260526_1400_add_trash_to_lots_devices.py`
P3	Audit before delete	Call `audit()` with the old values **before** `db.delete()`; the row is gone after commit. 242 `await audit(` calls in routers/services	`CLAUDE.md` Critical Rules, `services/audit_engine.py`
P4	`role_allowed()` for UI gating	Never hard-code a role list in a template. `role_allowed()` mirrors `require_roles` so custom roles (e.g. `trc_manager`) see the same buttons	`templates_config.py` `_role_allowed`, `.claude/skills/custom-role-permissions`
P5	Permission Matrix	Admin grants modules per role in Master Data; nav and backend both read the cached matrix	`auth/dependencies.py` `_matrix_grants_path`, `routers/master.py`
P6	AJAX partial refresh	Submit by fetch, parse the returned HTML with `DOMParser`, swap only the changed region, then re-bind handlers	`.claude/skills/ajax-partial-refresh`, commit `684fa27`
P7	Global Table	Heavy lists use server-side DataTables plus `global-table.js` (13 templates use `serverSide: true`)	`static/js/global-table.js`, e.g. `templates/devices/list.html`
P8	Server-side money	Totals calculated server-side with `Decimal`; priced records are snapshotted as columns	`.claude/skills/server-side-financial-snapshot`
P9	Schema provisioning at startup	`db_validator.validate_and_fix()` creates missing tables/columns before the app serves; aborts on unfixable issues	`db_validator.py`, `main.py` startup
P10	In-memory caches	Permissions, transitions, labels, dropdowns cached per process and refreshed every 60 s	`main.py` startup
P11	Error UX	One error page with a message and a "what to do" line; JSON for AJAX	`main.py` `_ERROR_INFO`

Drift found in code (for follow-up):

- `CLAUDE.md` says `is_trashed` is nullable and must be filtered with `isnot(True)`. The models now declare it `nullable=False`, and 126 filters use `== False` against 10 using `isnot(True)` (grep of `routers/`, `services/`). Both work on a NOT NULL column; the rule text is out of date.
- Only 4 templates call `role_allowed(`; 15 places still compare `current_user.role.value` directly (e.g. `templates/crm/contacts/list.html` l.52, `templates/dealers/list.html` l.54).

7. Folder structure map

Path	What it holds
`main.py`	App factory, middleware, router registration, error handlers, startup jobs
`config.py`	Settings from env vars, then `config.ini`, then defaults
`database.py`	Async engine, pool settings, `get_db`
`db_validator.py`	Schema validator and auto-fixer
`limiter.py`	SlowAPI limiter and key functions
`templates_config.py`	Shared `Jinja2Templates` and all Jinja globals
`auth/`	Staff, partner, care-agent and API-key auth
`routers/`	One module per feature area; `routers/api_v1/` for the JSON API
`models/`	SQLAlchemy models (57 modules)
`schemas/`	Pydantic schemas for the API
`services/`	Business engines (audit, events, cost, P&L, pricing, care ...)
`utils/`	Helpers (timezone, master data, grades, warranty, CSV decode ...)
`templates/`	Jinja pages by module; `base.html`, `error.html`
`static/`	`css/app.css`, `js/*.js`, `vendor/`, `m/telecalling/` PWA, `partner/` PWA, `manuals/`
`fieldops_app/`	Reliance FieldOps PWA, served behind login by `routers/fieldops.py`
`alembic/`	36 migration revisions
`migrate_*.py`, `backfill_*.py`, `seed_*.py`, `verify_*.py`	51 / 8 / 9 / 12 one-off scripts at repo root
`scripts/`	Backup/restore, Windows service install
`deploy/`	Linux: systemd units, nginx vhosts, auto-deploy and backup scripts, `DEPLOY.md`
`Dockerfile`, `docker-compose.yml`, `.dockerignore`	Container build used in production
`*.bat`, `*.ps1`, `*.iss`, `launcher.py`, `installer.py`	Windows / legacy run and installer tooling
`oxyqc-agent/`	IQC station hardware agent (EXE + source, PXE)
`care_agent_windows/`	Customer Care Agent source (service + tray)

Path	What it holds
`wa-service/`	Node WhatsApp service (integration disabled)
`tests/`	172 pytest files
`docs/` , `qa_docs/`	BRDs, plans/specs, audits, design system
`.claude/skills/`	7 project workflow skills
`.github/workflows/`	`deploy.yml` (self-hosted deploy), `build-agent.yml` (agent EXE)

Open questions

1. Is `.github/workflows/deploy.yml` still active? It rsyncs to `/opt/oxypc` and runs `systemctl restart oxypc`, but the Docker cutover disabled `oxypc.service` (docker spec §9). If a self-hosted runner is still registered it may conflict with the Docker auto-deploy.
2. Is there a public deployment at `app.oxypc.com` (named in `README.md` for `/fieldops`) or on Railway (mentioned in `main.py`, `limiter.py`, `deploy/oxypc-autodeploy.timer`)? Its host, owner and data are not in the repo.
3. Which ecosystem apps actually receive webhooks today, and which `OXYPC_ALLOWED_ORIGINS` are set in production?
4. Is the OxyQC standalone app (described in `CLAUDE.md` as offline-capable) still in use, or has it been replaced by the browser IQC page plus `oxyqc-agent`?
5. Is the Care Agent deployed to any customer yet? Its README says "not a shipped product".
6. nginx version on the Internal Server is not recorded.
7. Should `routers/repair.py.bak` and `wa-service/` be removed from the tree?

PART B

Data Model & Dictionary

This document describes how the OxyPC Inventory Lifecycle app stores its data. It is written from the SQLAlchemy models in `models/`, which are the real source of truth. `docs/schema.dbml` and `docs/databasetablesoxypcapp.pdf` are older snapshots (see section 6).

Every claim names the file it comes from. Where the code does not say something, this document says "not found" rather than guessing.

---

1. At a glance

Item	Value	Source
Database	PostgreSQL, async SQLAlchemy (`create_async_engine`)	`database.py`
ORM base	`database.Base` (`DeclarativeBase`)	`database.py`
Tables defined in `models/`	147 (`__tablename__` count)	`models/*.py`
Tables in `docs/schema.dbml`	89 `Table` blocks (file header still says "Tables: 57", generated 2026-04-29)	`docs/schema.dbml`
Separate FieldOps tables (own metadata, no FKs into OxyPC)	`fieldops_users` , `fieldops_records` , `fieldops_admin_audit`	`fieldops_db.py`
Primary keys	UUID (`UUID(as_uuid=True)` , `default=uuid.uuid4`) almost everywhere	`models/*.py`
Connection pool	`pool_size` 8, `max_overflow` 4 per worker (env `OXYPC_DB_POOL_SIZE` / `OXYPC_DB_MAX_OVERFLOW`)	`database.py`
Session pattern	one `AsyncSession` per request; commit on success, rollback on error	`database.py` `get_db()`

How the schema changes

The schema is changed in three different ways. All three are in use, which is the main reason the docs drift.

Mechanism	What it does	Source
Alembic	36 revisions, from `20260331_1600_..._initial_baseline.py` to `20261013_1200_dealers_sub_category_website_cid.py`	`alembic/versions/`
`db_validator.py` at startup	Creates missing tables (`create_all(checkfirst=True)`), adds missing columns (`ALTER TABLE ... ADD COLUMN IF NOT EXISTS`), seeds `stage_master` / `allowed_transitions`	`db_validator.py` (called from `main.py` startup)
One-shot `migrate_*.py` scripts	52 scripts at repo root, run by hand (e.g. `migrate_credit_note_workflow.py` , `migrate_work_orders_unique_constraint.py` , `migrate_perf_indexes.py`)	repo root

Many model comments say a column is "additive; auto-provisioned by db_validator" (for example `devices.entity` , `devices.stress_notes` , `spare_parts.crate_number`). Those columns have no Alembic revision.

2. Domains

Domain	What it covers	Main tables	Model files
Intake	Buying a lot, receiving it against an invoice (GRN), first inspection (IQC), creating the tag	`lots`, `lot_line_items`, `grn_imports`, `grn_lots`, `grn_stocked_tags`, `devices`, `iqc_inspections`, `crm_sourcing_deals`, `crm_purchase_orders`	`lot.py`, `grn_import.py`, `device.py`, `iqc_inspection.py`, `crm.py`
Inventory	Where each tag is, which stage it is in, how it moves	`devices`, `stage_movements`, `storage_locations`, `device_location_logs`, `buckets`, `stock_transfers`, `block_records`, `block_record_items`, `audit_records`, `audit_record_items`, `tag_number_movements`, `stage_master`, `allowed_transitions`, `device_aging`	`device.py`, `location.py`, `bucket.py`, `stock_transfer.py`, `block_tags.py`, `audit_record.py`, `stage_control.py`, `engines.py`
Repair	L1/L2 and L3/L4 work, work IDs, repair cost	`work_orders`, `repair_jobs`, `repair_attempts`, `device_costing`	`work_order.py`, `repair.py`, `engines.py`
QC	Stress test, legacy QC scoring, Final QC result	`stress_test_results`, `qc_checks`, final QC columns on `devices`, fail columns on `buckets`	`stress.py`, `qc.py`, `device.py`, `bucket.py`
Cosmetic	Paint line stages and who works each stage	`cosmetic_flow_rows`, cosmetic values of `devices.current_stage`	`cosmetic_flow.py`, `device.py`
Parts	Spare part master, stock, requests, PNA, parts GRN, part sales, estimates	`spare_parts`, `spare_parts_purchases`, `spare_parts_consumption`, `spare_parts_ledger`, `ram_tracking`, `part_requests`, `part_sourcing_requests`, `device_pna_parts`, `parts_grn`, `parts_grn_line_items`, `part_sale_requests`, `part_sales`, `part_estimates` (+3 line tables)	`spare_parts.py`, `engines.py`, `part_request.py`, `pna_part.py`, `parts_grn.py`, `part_sales.py`, `part_estimate.py`
Sales & CRM	Selling a tag, returns and credit notes, dispatch requests, warranty, contacts, deals, quotes, dealers, telecalling	`sales`, `returns`, `telecaller_dispatch_requests`, `companies`, `care_warranties`, `crm_contacts`, `crm_sales_opportunities`, `crm_quotes`, `dealers`, `dealer_orders`, `dealer_credit_notes`, `telecalling_records`	`sales.py`, `dispatch_request.py`, `company.py`, `care.py`, `crm.py`, `dealers.py`, `telecalling.py`
Finance	Money in and out, P&L inputs	`supplier_payments`, `customer_receipts`, payment columns on `crm_purchase_orders` / `crm_quotes`, `partner_bid_payments`, `device_costing`, `cost_config`, `business_pl_overrides`	`crm.py`, `partner_payment.py`, `engines.py`, `cost_config.py`, `business_pl_override.py`
Partner	Trade Partner (dealer) portal: listings, bookings, bids, payments	`partner_listings`, `partner_listing_devices`, `partner_bookings`, `partner_payment_proofs`, `partner_bids`, `partner_bid_documents`, `partner_bid_payments`, `lot_dealer_visibility`, `lot_booking_requests`, `partner_settings`, `partner_login_logs`	`partner.py`, `partner_payment.py`, `dealers.py`

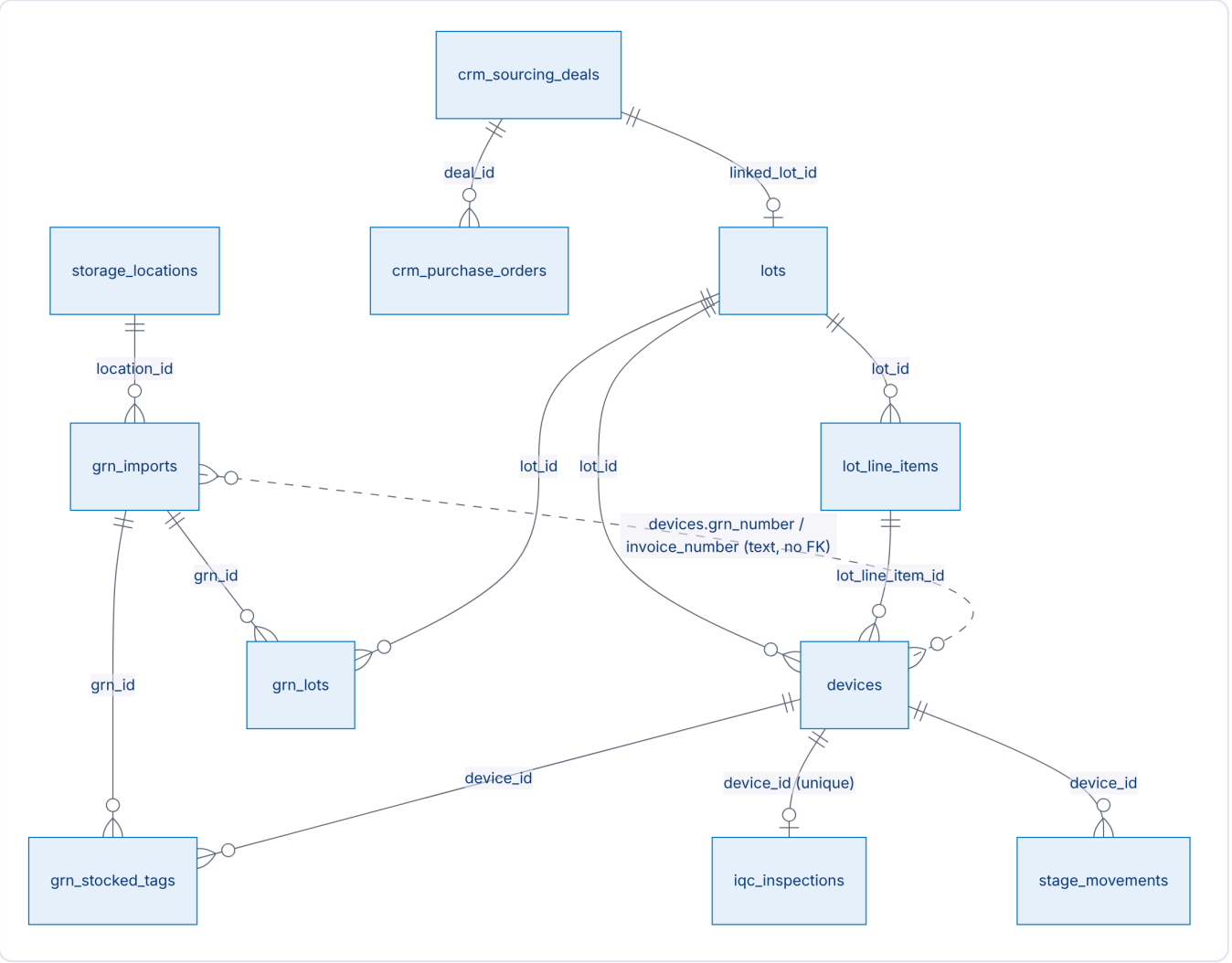
Domain	What it covers	Main tables	Model files
Admin / Audit	Users, roles, permissions, audit trail, events, API keys, settings	`users`, `login_logs`, `user_permissions`, `role_module_permissions`, `custom_roles`, `role_additional_permissions`, `audit_logs`, `event_log`, `webhooks`, `api_keys`, `app_settings`, `master_data`, `notifications`	`user.py`, `role_permissions.py`, `engines.py`, `event_log.py`, `webhook.py`, `api_key.py`, `settings.py`, `master.py`, `notification.py`

Not covered in detail here: QA/UAT tracking (`models/qa_uat.py`), Product Master / pricing (`models/product_master.py`), attendance (`models/attendance*.py`), Assign Leads (`crm_leads*` in `models/crm.py`), Customer Care tickets beyond warranty (`models/care.py`), and FieldOps (`fieldops_db.py`).

3. Entity-relationship diagrams

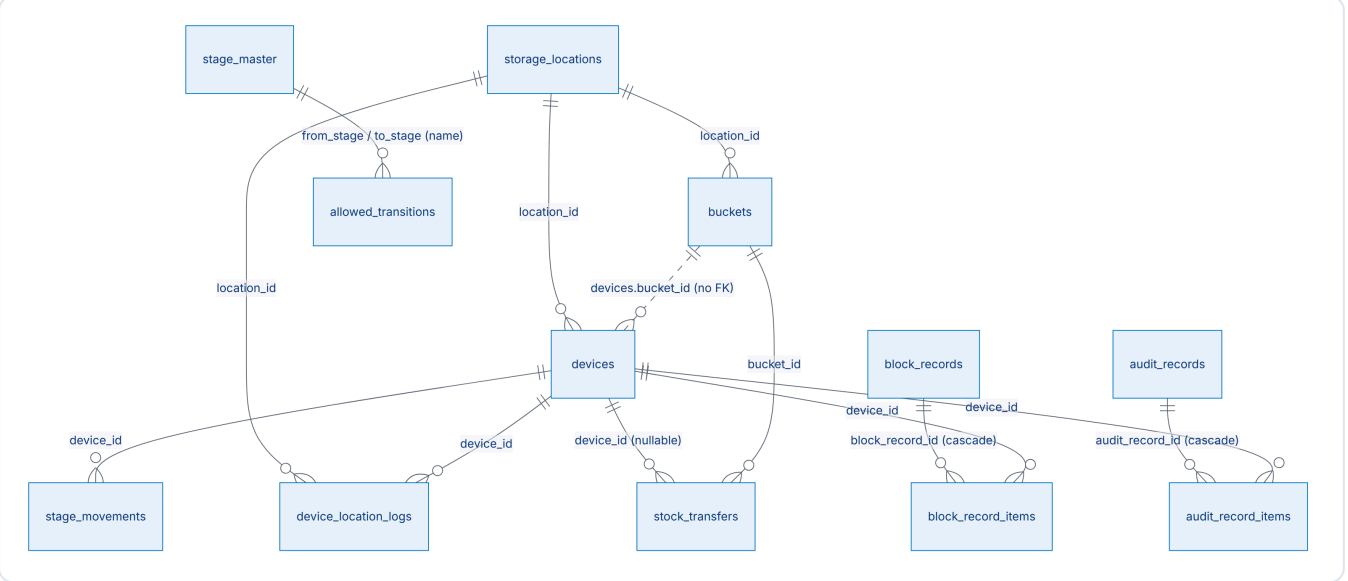
Solid lines are real foreign keys in the model. Dotted lines are links the code relies on but that have **no** foreign key (matched by a text value or an untyped UUID). Columns are left out to keep the diagrams readable; see section 5 for them.

3.1 Intake



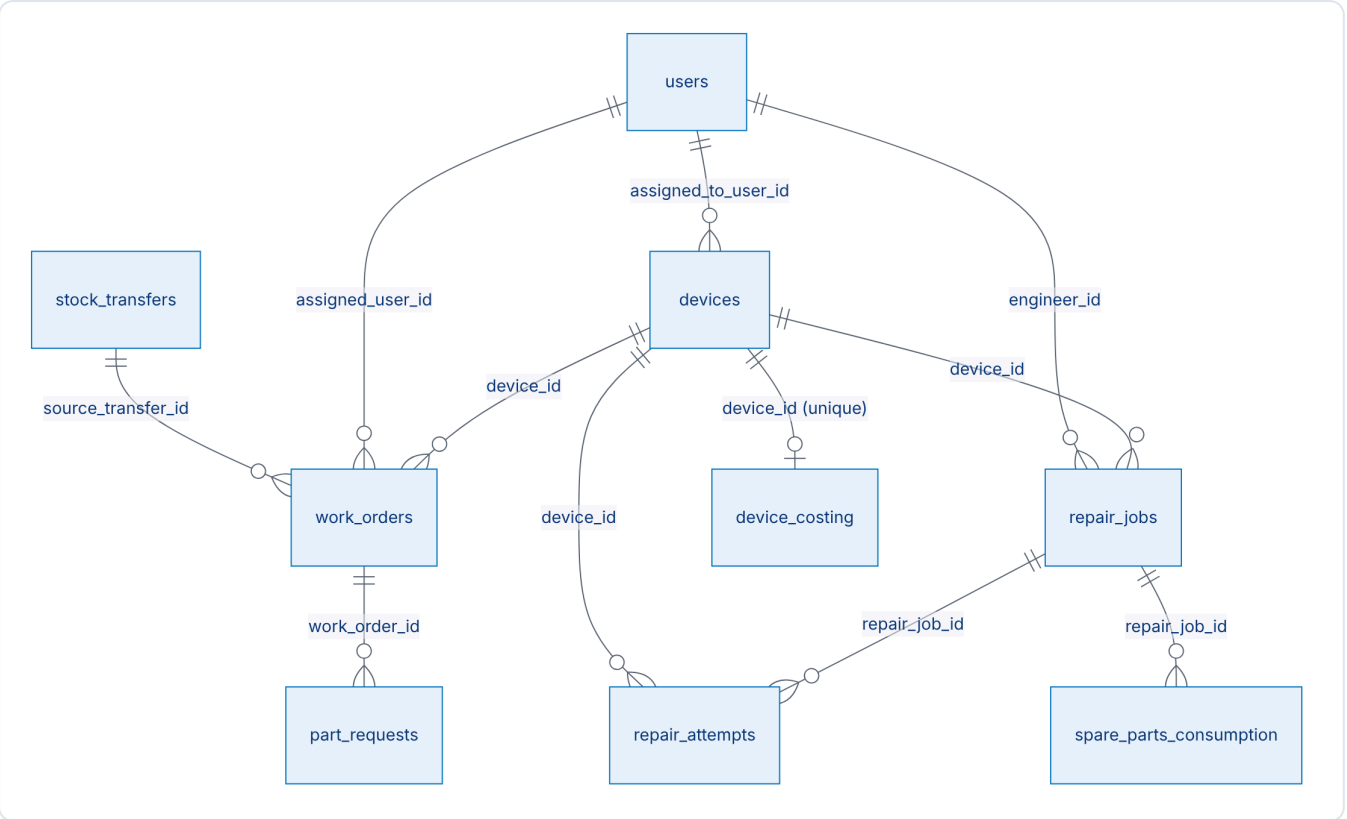
Sources: `models/crm.py`, `models/lot.py`, `models/grn_import.py`, `models/device.py`, `models/iqc_inspection.py`.

3.2 Inventory



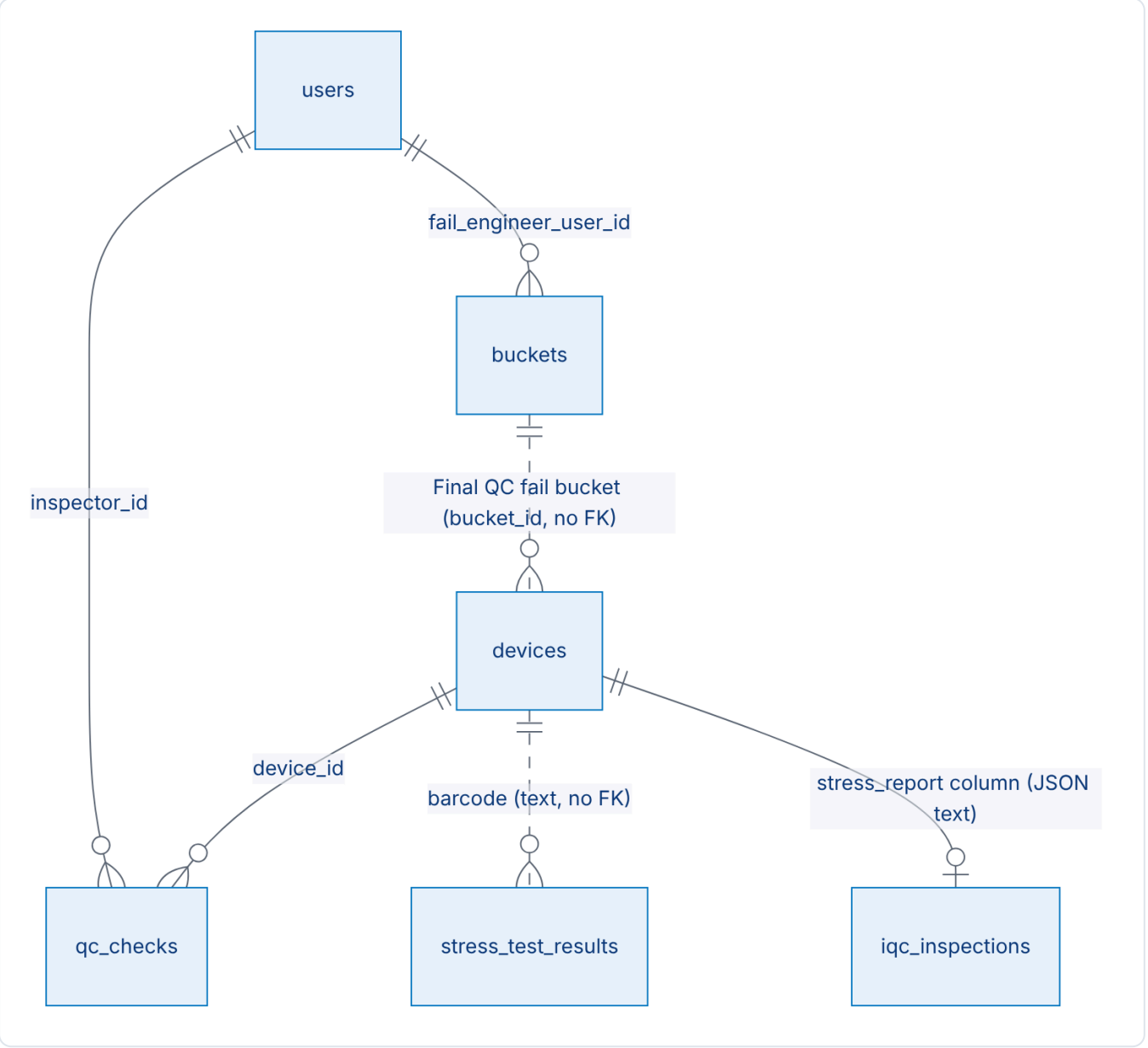
Sources: `models/device.py`, `models/location.py`, `models/bucket.py`, `models/stock_transfer.py`, `models/block_tags.py`, `models/audit_record.py`, `models/stage_control.py`.

3.3 Repair



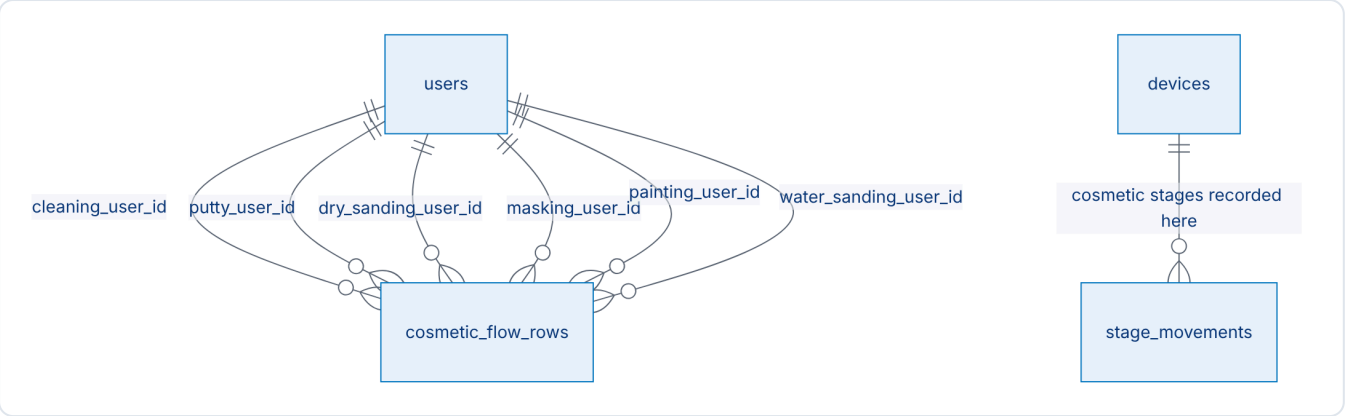
Sources: `models/work_order.py`, `models/repair.py`, `models/engines.py`, `models/spare_parts.py`, `models/part_request.py`.

3.4 QC and Final QC



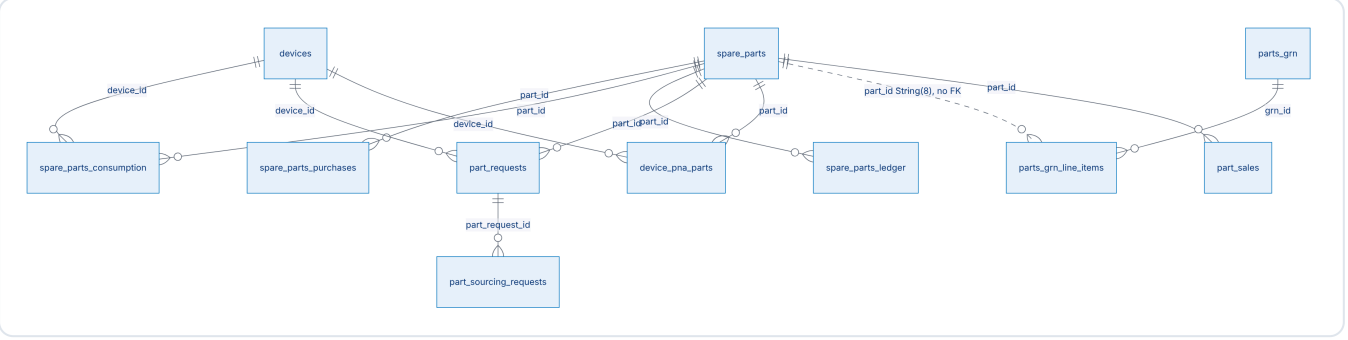
Final QC itself has no table. The result lives on `devices` (`final_qc_status`, `grade`, `fqc_*`, `touch_type`) and the stage moves to `final_qc_pass_hold` / `final_qc_fail_hold` (`models/device.py`). Sources: `models/qc.py`, `models/stress.py`, `models/bucket.py`.

3.5 Cosmetic



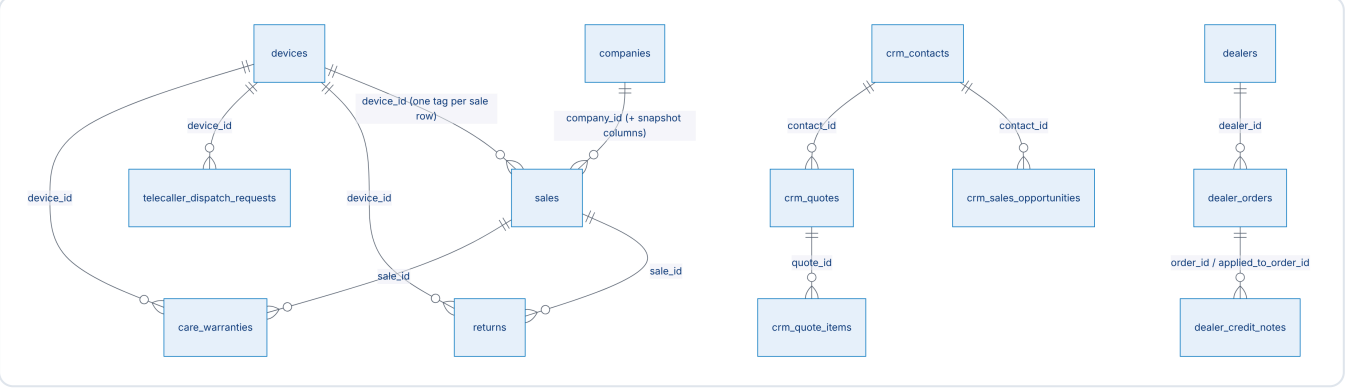
`cosmetic_flow_rows` is "purely a definition table today — nothing reads these rows yet to drive behaviour" (`models/cosmetic_flow.py` docstring). Progress through the paint line is tracked only through `devices.current_stage` and `stage_movements` (`COSMETIC_STAGES` in `models/device.py`).

3.6 Parts



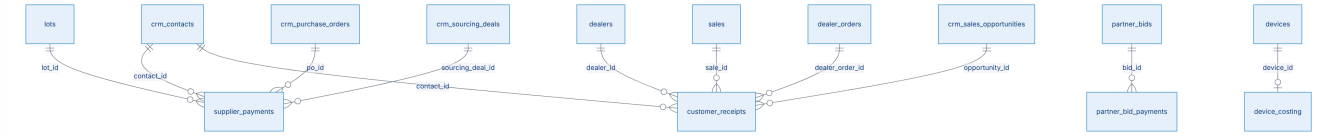
Sources: `models/spare_parts.py` , `models/engines.py` , `models/part_request.py` , `models/pna_part.py` , `models/parts_grn.py` , `models/part_sales.py` .

3.7 Sales, CRM and warranty



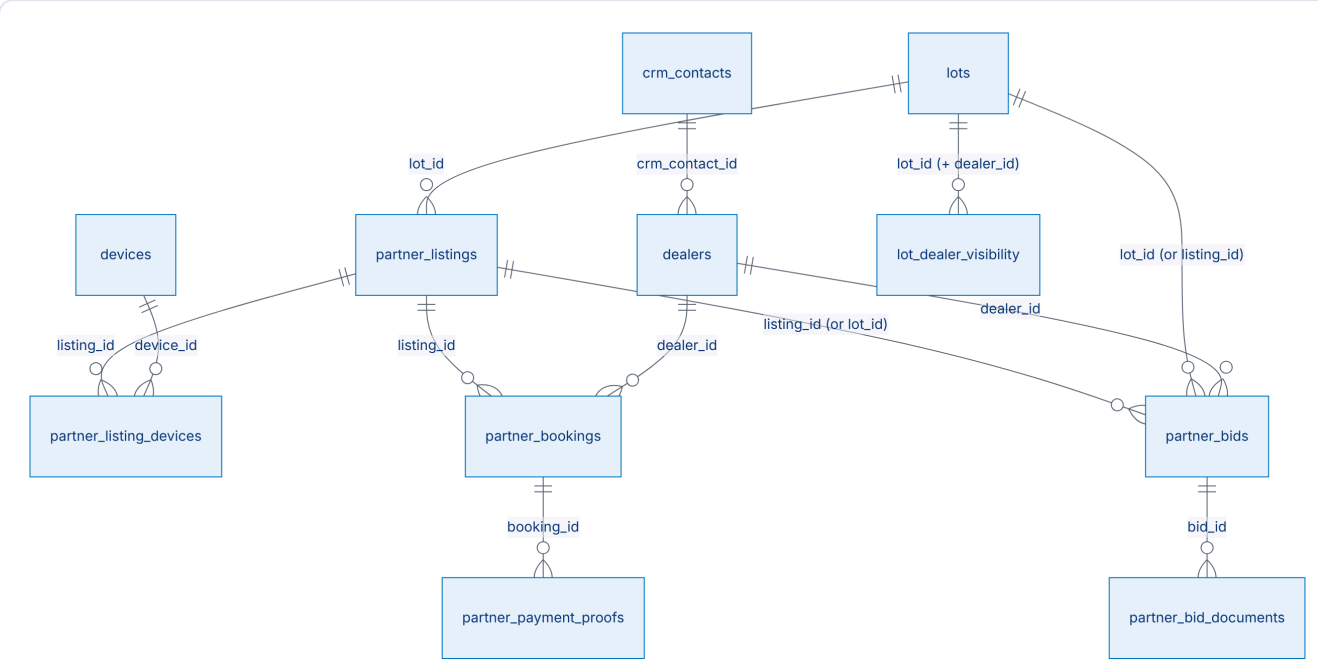
`crm_sales_opportunities.linked_sale_ids` is a comma-separated text list of sale UUIDs, not a FK (`models/crm.py`). Sources: `models/sales.py` , `models/company.py` , `models/dispatch_request.py` , `models/care.py` , `models/crm.py` , `models/dealers.py` .

3.8 Finance



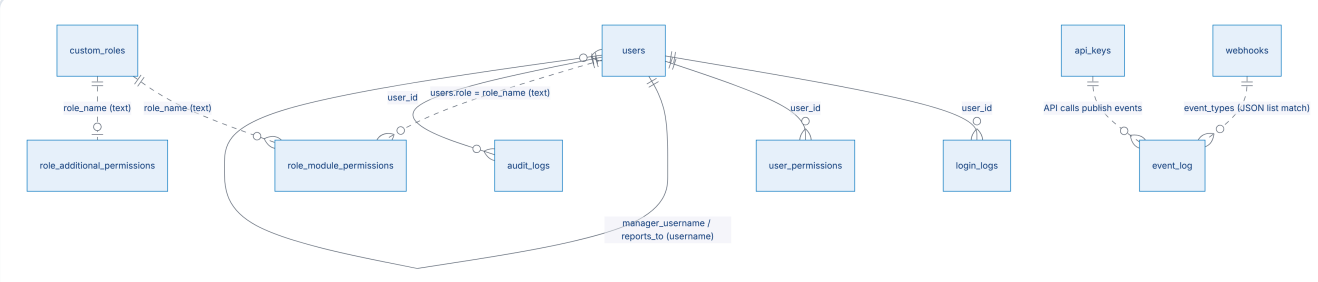
`cost_config` (key/value, `Numeric(10,2)`) and `business_pl_overrides` (year/month overrides) have no FKs. Sources: `models/crm.py`, `models/partner_payment.py`, `models/engines.py`, `models/cost_config.py`, `models/business_pl_override.py`.

3.9 Partner (Trade Partner portal)



`partner_bids.opportunity_id` points to `crm_sales_opportunities` on purpose without a FK ("a hard cross-context FK is exactly what the schema rules warn against", `models/partner.py`).

3.10 Admin and audit



Sources: `models/user.py`, `models/role_permissions.py`, `models/engines.py` (`AuditLog`), `models/event_log.py`, `models/webhook.py`, `models/api_key.py`.

4. Conventions

4.1 Naming

Convention	Detail	Source
Table names	snake_case, mostly plural (`devices` , `lots`). Exceptions: `parts_grn` , `device_costing` , `device_aging` , `event_log` , `cost_config` , `scrap_for_sale` , `partner_floor_config` , `attendance`	`models/*.py`
Primary key	`id` UUID. Exceptions: `stress_test_results.id` is an auto-increment Integer; `app_settings.key` and `partner_settings.key` are string PKs	`models/stress.py` , `models/settings.py` , `models/partner.py`
Tag Number	Stored as `devices.barcode` (String 100, unique). UI calls it "Tag Number"	`models/device.py`
Business numbers	`lot_number` ; `sale_number` = `SALE-####` from Postgres sequence `sale_number_seq` ; `work_id` 12 digits; `grn_imports.grn_number` 12 digits; `bucket_number` = `BKT` + 8 digits; `block_id` = `DDMMYYYY-CONTAINER-####` ; `CARE-0001` , `TPL-0001` , `TPB-0001` , `BID-0001` , `PO-2025-0001` , `SD-2025-0001` , `OPP-2025-0001` , `QT-2025-0001` , `CRM0001`	`routers/sales.py` (`nextval('sale_number_seq')`), `models/work_order.py` , `models/grn_import.py` , `models/bucket.py` , `models/block_tags.py` , `models/care.py` , `models/partner.py` , `models/crm.py`
"Who did it" columns	Usually a username string (`created_by` , `moved_by` , `sold_by` , `actioned_by`), not a FK to `users` . Some newer tables use real FKs (`block_records.created_by_user_id` , `audit_record_items.audited_by_user_id`)	`models/*.py`
Snapshots	Many tables copy values at the time of an action so later edits do not change history: `sales.company_*` , `block_record_items` device fields, `audit_record_items` device fields, `stock_transfers` device fields, `partner_bids.previous_amount/base_amount` , `partner_bookings.*_snapshot`	`models/sales.py` , `models/block_tags.py` , `models/audit_record.py` , `models/stock_transfer.py` , `models/partner.py`
Status columns	Mostly `String` with allowed values in a code comment, validated in routers/services. Only a few are real Postgres enums: `devicestage` , `devicegrade` , `repairstatus` , `blockitemstatus` , `zonetype` , `unittype` , `movementdirection` , plus location audit enums	`models/device.py` , `models/repair.py` , `models/block_tags.py` , `models/location.py`
Roles	`users.role` is free text via `RoleType` (String 60) so custom roles work; `UserRole` enum lists the 12 built-in roles	`models/user.py`

4.2 Soft delete

There is no single soft-delete pattern. Each table uses its own.

Table	Columns	Rule	Source
`lots`	`is_trashed` (Boolean, NOT NULL, default false), `trashed_at`	Filter with `is_trashed.isnot(True)` (project rule)	`models/lot.py` , `CLAUDE.md`
`devices`	`is_trashed` , `trashed_at` (Trash page) **and** `is_active` , `deleted_at` (deactivation)	Two separate flags. API v1 and stress save filter `is_active == True`	`models/device.py` , `routers/stress_api.py`

Table	Columns	Rule	Source
`spare_parts`, `parts_grn`	`is_trashed`, `trashed_at`		`models/spare_parts.py`, `models/parts_grn.py`
`scrap_for_sale`	`is_trashed`		`models/scrap_for_sale.py`
`crm_contacts`	`is_trashed`		`models/crm.py`
`grn_imports`	`is_deleted`, `deleted_at`	"GRN docs are kept + audit-logged, just hidden"; `grn_lots` rows are kept	`models/grn_import.py`
`dealers`	`trashed_at`, `trashed_by`	`trashed_at IS NULL` means live	`models/dealers.py`
`api_keys`, `webhooks`	`deleted_at` (+ `is_active`)		`models/api_key.py`, `models/webhook.py`
`device_pna_parts`	`is_active`	Unmarking sets false, never deletes	`models/pna_part.py`
`block_record_items`, `audit_record_items`, `lot_line_items`, `scrap_sale_items`, `as_is_lot_prices`	none	Hard-deleted with parent (`ondelete="CASCADE"` or ORM `delete-orphan`)	model files

Rule from `CLAUDE.md`: write an audit log entry before any `db.delete()`, because the row is gone after commit.

4.3 Money and decimals

Topic	Detail	Source
Column type	`Numeric(p, 2)`. Unit prices usually `Numeric(10,2)` or `(12,2)`; totals `Numeric(14,2)`; GST amounts on `lots` `Numeric(12,2)`, on `parts_grn` `Numeric(10,2)`	`models/lot.py`, `models/parts_grn.py`, `models/crm.py`
Currency	No currency column anywhere. All amounts are Indian Rupees by convention (₹ in templates, "Rs" in PDFs)	`templates/invoices/print.html`, `services/po_pdf.py`
Percentages	`Numeric(5,2)` / `(6,2)` (`min_margin_pct`, `floor_pct`)	`models/crm.py`, `models/partner.py`
Bid rounding	`Decimal`, rounded **up** to the whole rupee	`models/partner.py` `min_next_bid()`
Invoice GST maths	Done in Python `float` with `round(..., 2)`, not `Decimal`	`routers/invoices.py` `_compute_gst()`
PO PDF	Formats with `float(x)`	`services/po_pdf.py` `_rs()`
GST rates	`cost_config` keys `gst_rate_intra` / `gst_rate_inter`, default 18	`routers/invoices.py` `_fetch_gst_rates()`
Derived, not stored	`returns` balance, partner `balance_due`, `audit_records.difference`, partner payment status	`models/partner.py`, `models/audit_record.py`, `models/partner_payment.py`

The project's own skill note says priced documents should use "server-side Decimal totals" (`CLAUDE.md`). Invoice GST does not yet follow it.

4.4 Timestamps and time zone

Topic	Detail	Source
Default	`Column(DateTime, default=app_now)`, a **naive** datetime in the app time zone	`utils/timezone.py`
App time zone	`Asia/Kolkata` by default; changed at startup from `app_settings` and from Company Settings	`utils/timezone.py`
Time-zone-aware exceptions	`notifications.created_at` (`DateTime(timezone=True)`, `server_default=func.now()`) and `stress_test_results.run_at`	`models/notification.py`, `models/stress.py`
Date-only columns	`Date` for business dates: `grn_imports.grn_date/purchase_date`, `block_records.block_date`, `audit_records.report_date`, `supplier_payments.payment_date`, `devices.flag_*_date`	model files
Dates stored as text	`grn_imports.invoice_date` is `String(40)` (parsed from PDFs, "may be free-form")	`models/grn_import.py`
`updated_at`	Set by ORM `onupdate=app_now`; bulk `update()` statements do not trigger it	SQLAlchemy behaviour; models use `onupdate`

If the admin changes the time zone, older rows keep the old wall-clock values because no offset is stored.

5. Data dictionary

"Required" means `nullable=False` (or primary key) in the model. "App-required" means the column is nullable but the form or router insists on it. Types are SQLAlchemy types; `String(n)` is `varchar(n)`.

5.1 Lot — `lots` (`models/lot.py`)

Field	Type	Required	Meaning	Notes
id	UUID	PK	Row id	
lot_number	String(100)	Yes	Lot code, e.g. LOT-042	Unique, indexed. Test lots start `EPT-` (`models/master.py`)
supplier_name	String(200)	Yes	Supplier	
grn_system_number	String(50)	No	System GRN ref, e.g. GRN-178	
grn_number_new	Integer	No	Numeric GRN	
grn_date	DateTime	No	Material received date	
invoice_date	DateTime	No	Supplier invoice date	
invoice_no	String(100)	No	Supplier invoice no.	
invoice_value	Numeric(14,2)	No	Total invoice value	
taxable_amount	Numeric(14,2)	No	Taxable amount	
sgst / cgst / igst	Numeric(12,2)	No	GST amounts	
vehicle_number	String(30)	No	Vehicle	
e_way_bill	String(50)	No	Inbound e-way bill number (typed in)	Text only, not validated

Field	Type	Required	Meaning	Notes
po_number	String(100)	No	Our PO	
vendor_name	String(200)	No	Vendor from PO	May differ from supplier
buying_price	Numeric(12,2)	Yes	Total lot cost	
qty	Integer	Yes	Units in lot	
purchase_date	DateTime	Yes	Purchase date	
condition	String(30)	No	New / Refurbished / Used	
selling_price	Numeric(12,2)	No	Planned selling price	
notes	Text	No		
created_by	String(50)	No	Username	
created_at	DateTime	No		default `app_now`
is_trashed	Boolean	Yes	Soft delete	default false
trashed_at	DateTime	No	When trashed	
is_restricted	Boolean	Yes	Hide from partner catalog unless granted	default false; grants in `lot_dealer_visibility`

Child `lot_line_items`: `lot_id` (FK, required), `sub_category` (required), `brand`, `model`, `cpu`, `generation`, `ram_gb`, `has_ram`, `storage_gb`, `storage_type`, `has_storage`, `screen_size`, `grade` (A/B/C/D), `unit_price` (Numeric(12,2), required), `qty` (required, default 1), `notes`, `created_at`.

5.2 Device / Tag — `devices` (`models/device.py`)

One row per physical unit ("Tag Number").

Field	Type	Required	Meaning	Notes
id	UUID	PK		
barcode	String(100)	Yes	Tag Number	Unique, indexed
lot_id	UUID FK → lots	Yes	Source lot	
lot_line_item_id	UUID FK → lot_line_items	No	Invoice line it came from	
sub_lot_number	String(50)	No	As-Is sub-lot	Bulk-set by Change Floor / GRN Post-IQC
entity	String(50)	No	Owning company entity	e.g. OxyPC Computers / Renew Circuits; widened 30→50 on 2026-10-21
brand, model	String(50) / String(100)	No		brand indexed
device_type	String(30)	No		indexed
sub_category	String(20)	No	Laptop / Desktop / TFT	indexed
serial_no	String(100)	No		

Field	Type	Required	Meaning	Notes
invoice_number, po_number	String(100)	No	Invoice / PO on the tag	Replaced by sale invoice on sale (commit `eb48f29`, `253854f`)
grn_number	String(50)	No	GRN ref	indexed; text link to GRN
cpu, cpu_make, generation	String	No	Processor	
ram_gb, ram_type, ram_speed, ram_make, ram_summary	Integer / String	No	Memory	`ram_summary` packs several sticks
storage_gb, storage_type, hdd_capacity_gb, hdd_type, hdd_speed, hdd_summary	Integer / String	No	Storage	
total_ram_count, total_ram_size, total_hdd_count, total_hdd_size	String(50)	No	IQC counts	
battery_health_pct	Integer	No	0–100	
screen_size, color	String	No		
bios_password	Boolean	No		
grade	Enum `devicegrade`	No	Final grade	A, B, C, D, E, scrap
current_stage	Enum `devicestage`	Yes	Workflow stage	See enum below; default `iqc`; indexed
final_qc_status	String(10)	No	pass / fail	indexed
fqc_failure_reason, fqc_pass_notes, fqc_final_notes	Text	No	Final QC notes	
touch_type	String(10)	No	touch / non_touch	blank shown as NT
flag_authorized_by, flag_request_date, flag_target_date, flag_reason, flagged_at, flagged_by	String / Date / Text / DateTime	No	Urgent flag	Cleared when tag enters `stock_in` (event listener)
floor	String(50)	No	Zone value	
warehouse	String(100)	No	Legacy free text	Mirrors location name
location_id	UUID FK → storage_locations	No	Precise location	
bin_location	String(50)	No	Bin under location	Free text
bucket_id	UUID	No	Current bucket	**No FK**
assigned_to_user_id	UUID FK → users	No	Current owner employee	
l1l2_status	String(30)	No	New / Repair Started / Requested to L3/L4	default "New"
l34_status	String(30)	No	blank / Repair Started / Completed / Normal Scrap / Replacement Scrap	
l34_activity	String(100)	No	L3/L4 activity	Master Data list

Field	Type	Required	Meaning	Notes
repair_notes, stress_notes, notes	Text	No	Notes	
device_price	Numeric(12,2)	No	Unit buying price	
min_selling_price, max_selling_price	Numeric(12,2)	No	As-Is lot price band	
qty	Integer	No	Units this row covers	server default 1
return_status	Boolean	Yes	Returned via Process Return	default false
tag_return_status	String(120)	No	Return for Repair / Replaced by / Return for Credit Note	
replaced, replace_with_barcode	String	No	Replacement links	Tag text, no FK
scrap_verified	Boolean	Yes		default false
is_blocked	Boolean	Yes	Open Block Tags hold	Sell refused while true
partner_listed	Boolean	Yes	Backs a live partner listing	
is_active, deleted_at	Boolean / DateTime	Yes / No	Deactivation	
is_trashed, trashed_at	Boolean / DateTime	Yes / No	Trash	
created_at, updated_at	DateTime	No		`updated_at` indexed

`DeviceStage` values (in order): `grn`, `iqc`, `stock_in`, `l1`, `l2` (legacy, hidden from dropdowns), `l3`, `trc_production`, `qc_check` (labelled "Stress Test"), `cosmetic_received`, `cleaning`, `putty`, `dry_sanding`, `masking`, `painting`, `water_sanding`, `cosmetic_completed`, `final_qc`, `final_qc_pass_hold`, `final_qc_fail_hold`, `ready_to_sale`, `sold`, `returned`, `scrapped`, `scrap_for_sale`. `sold` is also hidden from generic stage pickers; only the sale flow sets it (`DROPDOWN_STAGES`).

Child `stage_movements`: `device_id` (FK, required), `from_stage`, `to_stage` (required, indexed), `moved_by`, `moved_at`, `exited_at`, `notes`.

5.3 IQC inspection — `iqc_inspections` (`models/iqc_inspection.py`)

One per device (`device_id` unique). Almost every check is `String(40)` holding a label.

Field	Type	Required	Meaning	Notes
id	UUID	PK		
device_id	UUID FK → devices	Yes	Inspected tag	Unique
inspector_name	String(100)	No		
inspected_at	DateTime	No		
power_on, bios_password, all_ok	String(40)	No	Functional status	Yes / No
status	String(50)	No	Power On / No Display / No Power	

Field	Type	Required	Meaning	Notes
screen_dot, screen_line, screen_functional, screen_discoloration, screen_broken, screen_flickering, screen_loose, screen_missing, screen_hinge_broken, screen_colour_spread, screen_keyboard_mark, screen_hard_press	String(40)	No	Screen checks	Yes / No
screen_patch	String(40)	No		No / Minor / Major
screen_scratch	String(40)	No		No / Minor Scratch / Major Scratch
touch_screen	String(40)	No		Yes / No / Yes - Faulty
panel_a_ , *panel_b_*, panel_c_, *panel_d_*	String(40)	No	Panels A (lid), B (base), C (bezel), D (palmrest): scratch, broken, missing, dent, colour_fade, rubber_cut (B only)	No / Yes / Major ...
keyboard_working, keyboard_colour_fade, keyboard_key_missing, keyboard_hard_press	String(40)	No	Keyboard	
speaker_status	String(50)	No		Both working / Both faulty / Left faulty / Right faulty
touchpad_working, touchpad_click_working, touchpad_scratch, touchpad_colour_fade, touchpad_missing, touchpad_logicboard	String(40)	No	Touchpad	
port_hdmi, port_usb_working, port_audio_jack	String(40)	No	Ports	Yes / Not Working / No
usb_a_ports, usb_c_ports, ethernet_ports	Integer	No	Port counts	
wifi_status	String(40)	No		Working / Faulty / Not Checked
webcam_status	String(40)	No		Ok / Faulty / Not Checked
hdd_connector, hdd_casing, battery_present, battery_cable, charging_port, dvd_drive	String(40)	No	Components	
cover_ram, cover_dvd, cover_storage, hinge_condition, hinge_cover	String(40)	No	Covers / hinge	
storage_health_pct, fan_sound_dba	Integer	No		
fan_working	String(40)	No		
r2v3_grade_category	String(40)	No	R2v3 category	C0 / C3 / C4 / C5
remarks	Text	No		
stress_report	Text	No	JSON from OxyQC stress run	Raw JSON string

5.4 GRN — `grn_imports`, `grn_lots`, `grn_stocked_tags` (`models/grn_import.py`)

Field	Type	Required	Meaning	Notes
id	UUID	PK		
grn_number	String(12)	Yes	12-digit GRN id	Unique, auto-generated
lot_number	String(60)	No	Lot text parsed from PDF	
lot_id	UUID FK → lots	No	First mapped lot	Legacy; `grn_lots` is source of truth since 2026-10-21
entity	String(50)	No	Owning entity	Backfilled to "OxyPC Computers" once
location_id	UUID FK → storage_locations	No	Default location for tags	Added 2026-10-21
invoice_number	String(100)	No		
invoice_date	String(40)	No		**Text**, not a date
sender_name	String(200)	No	"Vendor Name"	
quantity	Integer	No	Invoice quantity	
amount	Numeric(14,2)	No	Invoice price	
purchase_date, grn_date	Date	No		
po_number, vehicle_number, e_way_bill	String(50)	No		
notes	Text	No		
validated	Boolean	No	Validated via modal	
received_qty	Integer	No	Counted at validation	
validation_ref	String(100)	No		
validation_notes	String(500)	No		
source	String(20)	No	`invoice` / `post_iqc` / Manage Lots test value	
file_name, file_path	String	No	Uploaded invoice PDF	Under `uploads/grn`
file_hash	String(64)	No	SHA-256	indexed
created_by, created_at		No		
is_deleted, deleted_at	Boolean / DateTime	Yes / No	Soft delete	

`grn_lots`: `grn_id` (FK, required), `lot_id` (FK, required), unique (`grn_id`, `lot_id`), `created_by`, `created_at`. `grn_stocked_tags`: `grn_id`, `device_id`, `barcode_key` (upper-cased tag), `matched_invoice`, `source` (`invoice_match` / `grn_mapped` / `audit_recovery`), `created_at`; unique (`grn_id`, `barcode_key`); add-only.

5.5 Bucket — `buckets` (`models/bucket.py`)

Field	Type	Required	Meaning	Notes
id	UUID	PK		
bucket_number	String(30)	Yes	`BKT########`	Unique
name, location, category	String	No		`category` holds brand for normal buckets
status	String(20)	Yes		`stock_in` / `trc_pending` / `validated`
received_qty	Integer	No		
location_id	UUID FK → storage_locations	No		
assigned_to_production, _by, _at	Boolean / String / DateTime	Yes / No	Hand-off to production	
dept_assigned, _by, _at	Boolean / String / DateTime	Yes / No	Production Manager assigned	
fail_reason	String(30)	No	Final QC fail reason locked on first fail	
fail_engineer_user_id	UUID FK → users	No	Engineer for fail bucket	
fail_engineer_username, fail_engineer_name	String	No	Snapshot	
is_customer_return	Boolean	Yes	Created from Return Stock	
created_by, created_at, updated_at		No		

5.6 Work order / WorkID — `work_orders` (`models/work_order.py`)

Field	Type	Required	Meaning	Notes
id	UUID	PK		
work_id	String(12)	Yes	12-digit WorkID	Unique (enforced by `migrate_work_orders_unique_constraint.py`)
device_id	UUID FK → devices	Yes		
barcode	String(100)	No	Snapshot	
stage	String(5)	Yes	`l1` / `l2` / `l3`	
assigned_role	String(30)	No	e.g. `l1_engineer`	
assigned_user_id	UUID FK → users	No		
assigned_username, assigned_name	String	No	Snapshot	
status	String(20)	Yes		`pending` / `in_progress` / `completed`

Field	Type	Required	Meaning	Notes
requested_by_name	String(100)	No	L1/L2 engineer who requested L3/L4	
source_transfer_id	UUID FK → stock_transfers	No	Transfer that created it	
assigned_at	DateTime	Yes		
started_at, completed_at	DateTime	No		
paused_since	DateTime	No	Timeline paused (part out of stock)	
paused_days_total	Integer	Yes	Days paused so far	default 0
created_by, created_at		No		

Related `repair_jobs` (`models/repair.py`): `device_id`, `stage` (String(5)), `engineer_id` (FK users), `status` (enum `repairstatus`: `open` / `in_progress` / `completed`), plus L1/L2 fields (`faults`, `dust_cleaning`, `cmos_battery_change`, `thermal_paste`, `final_status` = Completed / PNA / Escalate to L4 / Scrap / Lot / Repair, `ram_*`, `hdd_*`) and L3 fields (`action_taken`, `problem_observed`, `scrap_reason`, `received_from`, `customer_internal`).

5.7 Part request — `part_requests` (`models/part_request.py`)

Field	Type	Required	Meaning	Notes
id	UUID	PK		
work_order_id	UUID FK → work_orders	No		
work_id	String(12)	No	Snapshot	
device_id	UUID FK → devices	Yes		
barcode	String(100)	No		
stage	String(5)	No	Page that raised it	`l1` / `l2` / `l3`
part_id	UUID FK → spare_parts	No		
part_name	String(150)	Yes	Label snapshot	
part_category, part_capacity, part_type, part_make, part_model	String	No		
request_type	String(20)	Yes		`new` / `replace` / `downgrade`
downgrade_type	String(20)	No	Only for downgrade	`normal` / `replace`
requested_by, engineer_name	String	No		
qty_requested	Integer	Yes		default 1
qty_handed_over	Integer	Yes		default 0

Field	Type	Required	Meaning	Notes
status	String(20)	Yes		`requested` / `handed_over` / `not_in_stock` / `procure` / `scrapped`
created_at, actioned_at, actioned_by		No		

`part_sourcing_requests` (same file): `status` `open` / `closed`; `source` `procure` / `production`; `verified`, `reupload_requested`, `confirmed` flags with `_at` / `_by`; `lot_id`, `estimate_id` FKs; `source_deal_id` is a String, not a FK.

5.8 PNA (Part Not Available) — `device_pna_parts` (`models/pna_part.py`)

Field	Type	Required	Meaning	Notes
id	UUID	PK		
device_id	UUID FK → devices	Yes		Unique with `part_name`
barcode	String(100)	No		indexed
part_name	String(150)	Yes		
part_category	String(100)	No		
part_id	UUID FK → spare_parts	No		
source	String(30)	Yes	Screen	`parts_consumption` / `l3l4`
marked_by, marked_at		No		
cleared_by, cleared_at		No		
is_active	Boolean	Yes	Still PNA	false = cleared, row kept

5.9 Spare part and Parts GRN — `spare_parts`, `parts_grn`, `parts_grn_line_items` (`models/spare_parts.py`, `models/parts_grn.py`)

`spare_parts`:

Field	Type	Required	Meaning	Notes
id	UUID	PK		
part_code	String(20)	Yes		Unique
name	String(150)	Yes		
category	String(30)	Yes		Options from `IQC_PART_CATEGORIES`
unit_price	Numeric(10,2)	Yes		default 0
qty_in_stock	Integer	Yes		Kept in sync with `spare_parts_ledger`
min_stock_alert	Integer	Yes		default 5
sold_qty	Integer	Yes	Sold as parts	Sellable = stock − handed over − sold
supplier, notes, source		No		

Field	Type	Required	Meaning	Notes
crate_number, make, model, part_lot	String	No		`part_lot` is free text, not a FK
part_type	String(20)	No	New / Replace / Upgrade / Downgrade	
created_at	DateTime	No		
is_trashed, trashed_at		Yes / No	Soft delete	

`parts_grn`: `grn_number` (String(20), unique, required); PO (`po_number`, `po_date`, `po_file`); invoice (`invoice_number`, `invoice_date`, `invoice_file`); e-way bill (`eway_bill_number`, `eway_bill_date`, `eway_bill_file`); vehicle (`vehicle_number`, `vehicle_seal_file`, `vehicle_image_file`); `status` (`in_plan` / `in_trc`, required); `date_received`, `vendor_name`; money (`invoice_value` Numeric(12,2), `sgst` / `cgst` / `igst` / `tax_amount` Numeric(10,2), `total_amount_invoice`); quantities (`total_po_qty`, `total_invoice_qty`, `total_physical_qty`); `created_by`, `created_at`, `is_trashed`, `trashed_at`.

`parts_grn_line_items`: `grn_id` (FK, required); `part_id` String(8) required (**not** a FK to `spare_parts.id`); `lot_number`, `po_number`, `grn_number`, `vendor_name`, `invoice_number`, `product_description`, `item_name`, `part_brand`, `part_model`, `part_name`, `invoice_qty`, `physical_qty`, `price` Numeric(10,2), `main_category`, `category`, `vehicle_number`, `invoice_ref`, `remarks`, `is_harvest` (Boolean, required), `created_at`.

5.10 QC / Stress report — `stress_test_results`, `qc_checks` (`models/stress.py`, `models/qc.py`)

`stress_test_results`:

Field	Type	Required	Meaning	Notes
id	Integer	PK		Auto-increment (not UUID)
barcode	String(100)	Yes	Tag Number	indexed; **no FK** to devices
brand	String(100)	No		
model_name	String(200)	No		
run_at	DateTime(tz)	Yes		Time-zone aware
duration	String(20)	No		
overall_status	String(30)	No		
results_json	JSON	No	Checklist `{item: "pass"/"fail"/"na"}` or automated run results	Items in `STRESS_CHECKLIST_ITEMS` (27 items); "Touch Screen" may be `na`
pdf_path	Text	No	Generated PDF	`static/stress_reports/` (`routers/stress_api.py`)
run_by	String(100)	No		

`qc_checks` (legacy scoring, still written by `routers/qc.py`): `device_id` (FK), `inspector_id` (FK users), `battery_score` / `screen_score` / `keyboard_score` / `body_score` (0–10), `total_score` (0–100), `result` (String(10), required, pass / fail), `grade` (A/B/C/D/S), `attempt_number` (required), `checked_at`, `notes`, `issues_found`, `send_to_stage`.

5.11 Cosmetic flow — `cosmetic_flow_rows` (`models/cosmetic_flow.py`)

Field	Type	Required	Meaning	Notes
id	UUID	PK		
label	String(100)	No	e.g. "Team A"	
cleaning_user_id, putty_user_id, dry_sanding_user_id, masking_user_id, painting_user_id, water_sanding_user_id	UUID FK → users	No	Person per stage	
created_by, created_at, updated_by, updated_at		No		

Not yet used to drive behaviour (model docstring). Only Admin and Cosmetic Manager can edit.

5.12 Final QC and grade — columns on `devices` (`models/device.py`)

Field	Type	Meaning	Notes
current_stage	Enum	`final_qc` → `final_qc_pass_hold` or `final_qc_fail_hold` → `ready_to_sale`	
final_qc_status	String(10)	`pass` / `fail`	
grade	Enum `devicegrade`	A, B, C, D, E, scrap	
touch_type	String(10)	`touch` / `non_touch`	
fqc_pass_notes	Text	Pass notes	
fqc_failure_reason	Text	Fail reason	Also locked onto `buckets.fail_reason`
fqc_final_notes	Text	Raw "Final Notes"	

There is no history table for Final QC decisions; only the latest value is kept. History is only in `stage_movements` and `audit_logs`.

5.13 Block Tag — `block_records`, `block_record_items` (`models/block_tags.py`)

`block_records`:

Field	Type	Required	Meaning	Notes
id	UUID	PK		
block_id	String(40)	Yes	`DDMMYYYY-CONTAINER-####`	Unique
block_date	Date	Yes		
container	Enum `unittype`	Yes		rack / crate / shelf / trolley / cabinet / floor
box_number	String(50)	No	Physical box	
location_zone	Enum `zonetype`	No		showroom, ground_floor, first_floor, second_floor, workshop, dispatch, warehouse, holding
block_for_user_id	UUID FK → users	No	Held for	

Field	Type	Required	Meaning	Notes
customer_name	String(150)	No		
created_by_user_id	UUID FK → users	Yes		
created_at	DateTime	No		

`block_record_items`: `block_record_id` (FK, cascade), `device_id` (FK), snapshot (`barcode` required, `lot_number`, `model`, `cpu`, `ram_gb`, `storage_gb`), `status` (enum `blockitemstatus`: `blocked` / `released`, required), `status_changed_by_user_id`, `status_changed_at`, `notes`, `created_at`. A tag sold from a block is closed as `released` with `notes` starting `"Sold — "` + sale number. While open, `devices.is_blocked` is true.

5.14 Sale — `sales` (`models/sales.py`)

Field	Type	Required	Meaning	Notes
id	UUID	PK		
sale_number	String(20)	Yes	`SALE-####`	Unique, from `sale_number_seq`
device_id	UUID FK → devices	Yes	The one tag sold	One row per tag
sale_price	Numeric(12,2)	Yes	GST-inclusive price	Invoice back-calculates tax
customer_name	String(100)	No		
customer_phone	String(20)	No		
customer_state	String(100)	No	Decides CGST+SGST vs IGST	
customer_address	Text	No		
invoice_no	String(50)	No	Typed invoice no.	Falls back to `sale_number` on print
payment_mode	String(50)	No	From Master Data `payment_mode`	
payment_reference	String(100)	No	Cheque / UTR	
sold_by	String(50)	No	Operator username	indexed
sales_person	String(100)	No	Person credited	Free text
sold_at	DateTime	No	Sale date	Admin can change (commit `8a0091b`)
notes	Text	No		
transport_mode	String(30)	No	courier / hand_delivery / self_pickup	
transport_via	String(100)	No	Courier name	

Field	Type	Required	Meaning	Notes
tracking_number	String(100)	No	AWB	
dispatch_date	DateTime	No		Set by Gate Pass exit
delivery_status	String(30)	No	pending / dispatched / delivered	
invoice_file_path	String(500)	No	Uploaded invoice PDF	`uploads/sale_invoices/`
warranty_type	String(20)	No	none / 30_days / 6_months / 1_year	default `none`
warranty_expires_at	DateTime	No	Computed server-side	`utils/warranty.py`
warranty_days	Integer	No	Running total after extensions	
sale_channel	String(20)	No	procurement / telecaller / showroom	
company_id	UUID FK → companies	No	Selling company	
company_name, company_address, company_gstin, company_state, company_state_code, company_phone, company_email	String	No	Snapshot for the legal invoice	Null on older sales (live fallback)

5.15 Gate Pass / Dispatch — no own table

Concept	Where it lives	Source
Telecaller request to sell	`telecaller_dispatch_requests`: `device_id` (FK, required), `barcode`, `telecaller_username`, `telecaller_name`, `qty_requested`, `qty_available`, `grade`, `status` (`requested` / `approved` / `rejected`), `rejected_notes/_at/_by`, `source` (`device` / `lot`), `model_name`, `model_make`, `created_at`, `approved_at`, `approved_by`	`models/dispatch_request.py`
Gate exit	`sales.delivery_status` set to `dispatched`, `sales.dispatch_date` stamped; audit actions `GATE_EXIT_CONFIRMED` / `GATE_EXIT_BULK_CONFIRMED`	`routers/gate_pass.py`
Delivery Challan	Rendered on demand from `sales` + `devices` + `lots`; nothing stored	`routers/invoices.py` `/invoices/waybill/{sale_id}`

5.16 Return / Credit Note — `returns`, `dealer_credit_notes` (`models/sales.py`, `models/dealers.py`)

`returns`:

Field	Type	Required	Meaning	Notes
id	UUID	PK		
sale_id	UUID FK → sales	Yes		

Field	Type	Required	Meaning	Notes
device_id	UUID FK → devices	Yes		
return_date	DateTime	No		
reason, condition_on_return	Text / String(50)	No		
action_taken	String(30)	No	restock / scrap / credit	
reentered_stage	String(50)	No	Stage the tag re-entered	Model comment: "iqc (default)"
processed_by	String(50)	No		
refund_amount	Numeric(12,2)	No	Paid back to customer	
notes	Text	No		
approval_status	String(20)	No	pending / approved / rejected	default pending
approved_by, approved_at, rejection_reason		No		
return_type	String(20)	No	customer / dealer	
serial_captured	String(100)	No		
warranty_status	String(20)	No	in_warranty / out_of_warranty / no_warranty	Server-computed
complaint_text	Text	No		
repair_cost, labour_cost	Numeric(12,2)	No	After repair	
customer_email, customer_name, customer_phone, customer_state, customer_address	String / Text	No	Also used as "Sender Details" for debit notes	
cn_number	String(50)	No	Old single-tag CN	No longer written
cn_stage	String(30)	No	Credit Note workflow stage	`Return Received` → `Debit Note Verified` → `Verify Payment` → `Payment Invoice Done` → `Restock Pending` → `CN Complete` (`CN_STAGES`)
debit_note_number	String(50)	No		
debit_note_amount	Numeric(12,2)	No		
payment_invoice	String(100)	No		

`dealer_credit_notes` : `credit_number` (unique, required), `dealer_id` (FK, required), `order_id` (FK), `credit_date` , `amount` (Numeric(14,2), required), `reason` , `items_description` , `notes` , `created_by` , `created_at` , `is_applied` (required), `applied_at` , `applied_to_order_id` (FK).

5.17 Payment — `supplier_payments`, `customer_receipts`, `partner_bid_payments` (`models/crm.py`, `models/partner_payment.py`)

`supplier_payments` (money out):

Field	Type	Required	Meaning	Notes
id	UUID	PK		
contact_id	UUID FK → crm_contacts	No	Supplier	
lot_id	UUID FK → lots	No		
po_id	UUID FK → crm_purchase_orders	No		
sourcing_deal_id	UUID FK → crm_sourcing_deals	No		
payment_date	Date	Yes		default today
amount	Numeric(14,2)	Yes		
payment_mode	String(20)	No	cash / upi / neft / cheque / rtgs	
reference_no	String(100)	No	UTR / cheque	
is_advance	Boolean	No		
notes, invoice_path, payment_photo_path		No	Files under `uploads/`	
created_by, created_at		No		

`customer_receipts` (money in): same shape with `dealer_id`, `sale_id`, `dealer_order_id`, `opportunity_id` FKs, `receipt_date` (Date, required), `amount` (required).

`partner_bid_payments`: `bid_id` (FK, required), `payment_date`, `payment_mode` (`bank_transfer` / `upi`), `payment_utr`, `payment_amount` (Numeric(14,2)), `notes`, `submitted_by`, `submitted_at`, `verified` (required), `verified_by`, `verified_at`, `verify_notes`. Status shown to users is derived, never stored: PO Pending → Payment Pending → Payment Verifying → Payment Done (`payment_status()`).

`crm_purchase_orders` and `crm_quotes` also carry `payment_status` (pending / paid), `utr_number`, `payment_snapshot_path`, `invoice_path`.

5.18 Warranty — `sales` warranty columns and `care_warranties` (`models/sales.py`, `models/care.py`)

Two warranty sources exist.

Source	Fields	Notes
`sales`	`warranty_type`, `warranty_expires_at`, `warranty_days`	Durations in `utils/warranty.py`: none, 30 days, 182 days, 365 days. Used by Extended Warranty page

Source	Fields	Notes
`care_warranties`	`device_id` (FK, required), `sale_id` (FK), `policy_id`, `coverage_type` (main_device / battery / adapter / accessory), `start_event` (invoice_date / delivery_date / replacement_dispatch), `start_date`, `expiry_date`, `battery_expiry_date`, `status` (required; `not_started`, `active`, `expiring`, `expired`, `void`, `suspended`, `replaced`, `returned`, `bought_back`), `extension_reference`, `replacement_reference`, `void_reason`, `created_at`, `updated_at`	Read by `services/care_service.py` `resolve_warranty()`. No code in `routers/` or `services/` creates rows (grep for `CareWarranty(` finds none)

5.19 User — `users` (`models/user.py`)

Field	Type	Required	Meaning	Notes
id	UUID	PK		
username	String(50)	Yes	Login	Unique (`uq_users_username`)
full_name	String(100)	Yes		
password_hash	String(200)	Yes		
role	RoleType (String(60))	Yes	Built-in or custom role	default `sales`
status	Boolean	No	Active	`is_active` property
email	String(150)	No		
whatsapp_number	String(20)	No	Former WhatsApp module	
designation	String(100)	No	Job title	
reports_to	String(50) FK → users.username	No	Org chart	
manager_username	String(50) FK → users.username	No	Attendance / team rollups	
entity	String(50)	No (app-required)	Company entity	Added 2026-10-17
tenant	String(50)	No	default `oxypc_internal`	Multi-tenant placeholder
created_at, created_by, last_login		No		

Built-in roles (`UserRole`): admin, inventory_manager, iqc_inspector, l1_engineer, l2_engineer, l3_engineer, qc_inspector, sales, spare_parts_manager, telecaller, sales_manager, cosmetic_manager.

5.20 Role / Permission — `role_module_permissions`, `custom_roles`, `role_additional_permissions`, `user_permissions` (`models/role_permissions.py`, `models/user.py`)

Table	Fields	Notes
`role_module_permissions`	`role_name` (String(60), required), `module` (String(60), required), `can_enable`, `can_add`, `can_edit`, `can_upload`, `updated_at`, `updated_by`	Unique (`role_name`, `module`). Admin bypasses all checks
`custom_roles`	`role_name` (unique), `display_name`, `created_by`, `created_at`	
`role_additional_permissions`	`role_name` (unique) + `can_upload`, `can_download`, `can_export`, `can_print`, `can_add_new_data` (legacy), `can_view_pricing`, `can_set_price`, `can_add_lot`, `can_add_iqc`, `can_add_grn`, `can_edit_devices`, `can_edit_iqc`, `can_edit_lot`, `can_edit_grn`, `updated_at`, `updated_by`	All Boolean, default true
`user_permissions`	`user_id` (FK), `permission` (String(100)), `granted`, `granted_by`, `granted_at`	Per-user flags

Permissions are cached in memory (`_PERM_CACHE`, `_ADDITIONAL_PERM_CACHE`) and refreshed when an admin saves the matrix.

5.21 Audit record and event log — `audit_logs`, `event_log`, `audit_records` (`models/engines.py`, `models/event_log.py`, `models/audit_record.py`)

`audit_logs` (the audit trail):

Field	Type	Required	Meaning	Notes
id	UUID	PK		
user_id	UUID FK → users	No		
username	String(50)	No		
action	String(50)	Yes	e.g. STAGE_MOVED, SALE_CREATED, GATE_EXIT_CONFIRMED	
table_name	String(50)	No		
record_id	String(50)	No		
old_value, new_value	Text	No	JSON snapshots	
ip_address	String(45)	No		
notes	Text	No		
timestamp	DateTime	Yes		indexed

Written by `services/audit_engine.py` `audit()`. The model says "No UPDATE or DELETE permitted", but this is a convention; no database trigger or grant enforcing it was found.

`event_log` (integration events): `event_type` (String(50), required), `payload` (JSON, required), `source_module`, `published_at` (required), `webhook_attempts` (required), `last_attempt_at`, `last_status_code`. Written by `services/webhook_dispatcher.py`.

`audit_records` / `audit_record_items` are **physical stock counts**, not an audit trail. `audit_records`: `report_date` (required), `my_stage` (enum), `my_location_id`, `category`, `current_count`, `physical_count`, `notes`. Items snapshot the device and compare `system_stage` / `system_location_id` with `my_stage` / `my_location_id`, plus `audited_by_user_id` (required).

Other audit-like tables: `care_audit_logs` (Care module), `login_logs`, `partner_login_logs`, `stage_movements`, `device_location_logs`, `fieldops_admin_audit`.

---

6. Gaps and inconsistencies

6.1 `docs/schema.dbml` versus models

`docs/schema.dbml` was generated on 2026-04-29 and last changed in commit `fac334c`. `docs/databasetablesoxypcapp.pdf` is a jsPDF export dated 29 Apr 2026 of the same diagram. Both are well behind the code.

Gap	Detail
Tables missing from dbml (58)	`api_keys`, `as_is_lot_prices`, `attendance_groups`, `attendance_group_members`, `audit_records`, `audit_record_items`, `block_records`, `block_record_items`, `buckets`, `business_pl_overrides`, `cosmetic_flow_rows`, `cost_config`, `crm_contact_locations`, `crm_contact_team_members`, `crm_leads`, `crm_lead_groups`, `crm_lead_calls`, `custom_roles`, `deals`, `deal_items`, `dealer_quotations`, `dealer_quotation_items`, `device_pna_parts`, `event_log`, `grn_imports`, `grn_lots`, `grn_stocked_tags`, `hw_ram`, `hw_storage`, `model_requests`, `notifications`, `part_estimates`, `part_estimate_lines`, `part_estimate_checklist_lines`, `part_estimate_open_lines`, `part_sale_requests`, `part_sales`, `partner_bids`, `partner_bid_documents`, `partner_bid_payments`, `parts_grn_line_items`, `pm_products`, `pm_dropdown_values`, `pm_competitive_lists`, `pm_competitor_prices`, `pm_calc_uploads`, `pm_calc_rows`, `role_module_permissions`, `role_additional_permissions`, `scrap_for_sale`, `scrap_sales`, `scrap_sale_items`, `stock_validations`, `stress_test_results`, `tag_number_movements`, `telecalling_assignments`, `tickets`, `webhooks`
Tables with column drift (26)	Largest: `devices` (38 columns missing from dbml, e.g. `entity`, `bucket_id`, `is_blocked`, `location_id`, `fqc_*`, `flag_*`), `sales` (20, incl. all `company_*` snapshot and warranty columns), `returns` (20, incl. the whole Credit Note workflow), `crm_contacts` (19 KYC/consent columns), `dealer_calls` (15), `iqc_inspections` (14), `telecalling_records` (13), `part_sourcing_requests` (13), `parts_grn` (10), `spare_parts` (9), `crm_purchase_orders` (8), `stock_transfers` (7), `part_requests` (7), `users` (6), `dealers` (6), `lots` (5, incl. `is_trashed`)
Columns in dbml but not in models	None found
Header count	dbml header says "Tables: 57" but contains 89 `Table` blocks

Several model comments say a change was "approved via dbdiagram" (for example `grn_lots`, `grn_stocked_tags`, `grn_imports.entity`, `grn_imports.location_id` on 2026-10-21, `users.entity` on 2026-10-17), but the approved diagram was not saved back to `docs/schema.dbml`. `CLAUDE.md` requires dbdiagram approval for schema changes, so the repo copy should be refreshed after each approval.

6.2 Missing or soft foreign keys

Column	Points to	Problem
`devices.bucket_id`	`buckets.id`	Plain UUID, no FK (`models/device.py`)
`stress_test_results.barcode`	`devices.barcode`	Text match only (`models/stress.py`)
`parts_grn_line_items.part_id`	spare part	`String(8)` , not a FK (`models/parts_grn.py`)
`part_sourcing_requests.source_deal_id`	`crm_sourcing_deals.id`	String(50) (`models/part_request.py`)
`crm_activities.deal_id`	sourcing deal or sales opportunity	Polymorphic, typed by `deal_type` (`models/crm.py`)
`crm_sales_opportunities.linked_sale_ids`	`sales.id`	Comma-separated text (`models/crm.py`)
`partner_bids.opportunity_id`	`crm_sales_opportunities.id`	Deliberately no FK (`models/partner.py`)
`devices.grn_number` , `devices.invoice_number`	`grn_imports`	Text match (`models/device.py` , `services/grn_stocked.py`)
`devices.replaced` , `replace_with_barcode`	other device	Tag text
`role_module_permissions.role_name`	`users.role` / `custom_roles.role_name`	Text
Most `*_by` columns	`users.username`	Text, survives user deletion but not renames

6.3 Other inconsistencies

Topic	Detail	Source
Three stage lists	`DeviceStage` has 24 values. `STAGE_SEQUENCE` / `DEFAULT_TRANSITIONS` in `models/stage_control.py` and `CANONICAL_STAGES` / `CANONICAL_TRANSITIONS` in `db_validator.py` each list fewer (both omit `putty` , `final_qc_pass_hold` , `final_qc_fail_hold` , `scrap_for_sale` ; `stage_control.py` also omits `grn` , `cosmetic_received` , `cosmetic_completed`)	`models/device.py` , `models/stage_control.py` , `db_validator.py`
Grade scales	`devices.grade` A–E + scrap; `qc_checks.grade` A/B/C/D/S; `lot_line_items.grade` A/B/C/D; CRM `GRADES` A, B, C, D, Mix, As-Is, Scrap	`models/device.py` , `models/qc.py` , `models/lot.py` , `models/crm.py`
Two soft-delete flags on devices	`is_trashed` and `is_active` mean different things; queries must check the right one	`models/device.py`
`CLAUDE.md` says `lots.is_trashed` is nullable	Model has `nullable=False, server_default='false'` . The `isnot(True)` rule is still safe	`models/lot.py` , `CLAUDE.md`
Two warranty stores	`sales.warranty_*` and `care_warranties` ; the second has no writer	`models/sales.py` , `models/care.py`
Legacy columns kept	`crm_contact_numbers.contact_id` , `returns.cn_number` , `role_additional_permissions.can_add_new_data` , `devices.warehouse` , `grn_imports.lot_id` , unused System Stage/Location columns on `audit_records`	model comments

Topic	Detail	Source
Repair stage field width	`work_orders.stage` and `repair_jobs.stage` are `String(5)` ; they hold only `l1` / `l2` / `l3` , while `devices.current_stage` uses the full enum. Any future repair stage name longer than 5 characters (e.g. `trc_production`) would not fit	`models/work_order.py` , `models/repair.py`
Indexes	`migrate_perf_indexes.py` adds `ix_device_location_logs_location_id` and drops the duplicate `idx_devices_location_id` . Other indexes come from `index=True` , Alembic `20260426_0940_..._add_missing_fk_indexes.py` , `20260427_1811_..._add_composite_indexes_sprint18.py` , `20260925_1200_add_dashboard_perf_indexes.py` , and model `__table_args__` (for example the partial index `ix_dealer_orders_overdue`)	listed files

6.4 Recent schema changes (newest first)

Date (from code comments / revision names)	Change	Source
2026-10-21	`grn_lots` (many-to-many GRN↔Lot), `grn_stocked_tags` , `grn_imports.entity` , `grn_imports.location_id` ; `devices.entity` widened to 50	`models/grn_import.py` , `models/device.py`
2026-10-17	`users.entity`	`models/user.py`
2026-10-16	`devices.bin_location`	`models/device.py`
2026-10-13	`dealers.dealer_sub_category` , `website` , `google_cid`	`alembic/versions/20261013_1200_dealers_sub_category_website_cid.py`
2026-10-10	`devices.touch_type` , `devices.flag_*` , `block_records.box_number` ; stress checklist items added	`alembic/versions/20261010_*` , `models/stress.py`
2026-10-07	`as_is_lot_prices`	`alembic/versions/20261007_1000_add_as_is_lot_prices.py`
2026-10-01	`block_records` , `block_record_items` ; `audit_records.my_*`	`alembic/versions/20261001_*`
2026-09-30	`audit_records` , `audit_record_items`	`alembic/versions/20260930_1200_add_physical_audits.py`
2026-09-22 to 09-24	Credit Note workflow on `returns` , `devices.tag_return_status/sub_lot_number` , `min/max_selling_price` , `stock_transfers.source` , `users.designation/reports_to` , unique `work_orders.work_id`	`migrate_credit_note_workflow.py` , `migrate_return_creditnote_sublot_fields.py` , `migrate_as_is_lot_selling_price.py` , `migrate_stock_transfer_source.py` , `migrate_user_designation_reports_to.py` , `migrate_work_orders_unique_constraint.py`

PART C

Integrations

This document lists every way data enters or leaves the OxyPC Inventory Lifecycle app: printed documents, machine APIs, desktop agents, partner and mobile front ends, file imports and exports, and messaging. Each section says what is live, what is partial, and what is missing. Every claim names the file it comes from.

Owner in this document means the role that the code lets operate the interface. No technical owner is named anywhere in the code; the only named person is Pankaj (Director, gives feature requests) in `CLAUDE.md`. Assigning a technical owner per interface is an open action.

1. Summary

#	Interface	Direction	Status
2.1	Sale Tax Invoice (HTML print)	Out (document)	Live, partial (see gaps)
2.2	Delivery Challan (HTML print)	Out (document)	Live
2.3	E-way bill generation	Out (government portal)	**Not implemented**
2.4	GST e-invoice (IRN / QR)	Out (government portal)	**Not implemented**
2.5	Gate Pass (exit confirmation)	Internal	Live
2.6	Ready to Dispatch / telecaller requests	Internal	Live
2.7	Purchase Order and Quote PDFs	Out (document)	Live
2.8	Invoice PDF upload and parsing (GRN, Sale)	In (file)	Live, best-effort
3.1	Session JSON API `/api/*`	Browser ↔ server	Live
3.2	Machine API `/api/v1/*` (API keys)	In / out	Live
3.3	Outbound webhooks + `event_log`	Out	Live
4.1	OxyQC IQC machine API `/iqc/api/*`	In	Live (client source not in repo)
4.2	OxyQC Diagnose agent (`127.0.0.1:8765`)	Station → browser	Live
4.3	Stress test runs and on-device results	In	Live
4.4	Customer Care Agent API `/care/api/v1/*`	In / out	Partial (pilot; agent not shippable)
5.1	Trade Partner portal `/partner/*`	In / out	Live
5.2	Partner payment verification `/partner-payments`	Internal	Live
5.3	Mobile telecalling PWA `/m/telecalling`	In / out	Live
5.4	FieldOps standalone app `/fieldops`	Separate app	Live only when `FIELDOPS_DATABASE_URL` is set

#	Interface	Direction	Status
6.1	WhatsApp bridge (`wa-service`)	Out	**Removed** (residual call left in Care)
6.2	`wa.me` deep links	Out (user's phone)	Live
6.3	Email / SMTP	Out	**Not found**
7.1	Bulk CSV upload `/bulk-upload`	In (file)	Live
7.2	Other CSV / Excel imports	In (file)	Live
7.3	Report exports (CSV / Excel)	Out (file)	Live
7.4	Daily Stock Report (scheduled files)	Out (file)	Live
7.5	File uploads and `/uploads` static mount	In / out	Live, with a security gap

2. Documents, tax and dispatch

2.1 Sale Tax Invoice

Item	Detail
Purpose	Printable "TAX INVOICE" for one sale
Direction	Out (HTML page the user prints from the browser)
Protocol / format	HTTP GET, HTML (`templates/invoices/print.html`)
Endpoint	`GET /invoices/print/{sale_id}` (`routers/invoices.py`)
Auth	Staff session cookie (`get_current_user` , `auth/dependencies.py`)
Data fields	From `sales` : `sale_number` , `invoice_no` (falls back to `sale_number`), `sale_price` , `customer_name/phone/state/address` , `company_*` snapshot. From `devices` : `brand` , `model` , `cpu` , `ram_gb` , `storage_gb` , `storage_type` , `barcode` , `serial_no` . From `lots` . GST rates from `cost_config` keys `gst_rate_intra` / `gst_rate_inter` (default 18)
Tax logic	`sale_price` is treated as GST-inclusive. Same state (customer vs company, string compare) → CGST + SGST split in half; otherwise IGST (`_compute_gst()`). Missing `customer_state` defaults to `"Delhi"`
Error handling	Unknown sale → redirect to `/sales?error=Sale+not+found`
Owner	Any logged-in user can open it; sales are created by sales roles (`routers/sales.py`)
Status	**Live, partial**

Gaps found in the code:

Gap	Source
One tag per invoice: the template prints a single line with qty 1. Multi-tag sales share an `invoice_no` but print separately	`templates/invoices/print.html` , `models/sales.py` (`device_id` per row)
HSN code hard-coded to `8471`	`templates/invoices/print.html`
GST calculated with Python `float` , not `Decimal`	`routers/invoices.py` `_compute_gst()`
`invoice_no` is free text typed by the user; no sequential, gap-free invoice series per financial year	`routers/sales.py` (`invoice_no: str = Form("")`)

Gap	Source
No PDF is generated or stored; the user prints the HTML. A separately uploaded invoice PDF can be attached (`sales.invoice_file_path`)	`routers/sales.py` `/sales/parse-invoice-pdf`
No IRN, no signed QR code, no e-invoice fields	grep for `irn` / `einvoice` finds nothing
Older sales without a company snapshot use the live "oldest active company"	`routers/invoices.py` `_sale_company_snapshot()` , `routers/settings.py` `get_company_settings`

2.2 Delivery Challan (waybill)

Item	Detail
Purpose	"DELIVERY CHALLAN" that travels with the goods; used by Gate Pass
Direction	Out (HTML print)
Endpoint	`GET /invoices/waybill/{sale_id}` (`routers/invoices.py` , `templates/invoices/waybill.html`)
Auth	Staff session
Data fields	Same sale / device / lot / company data as the invoice, plus signature boxes for dispatcher and receiver
Error handling	Unknown sale → redirect with error
Owner	Sales and Gate Pass users
Status	**Live**

2.3 E-way bill

Item	Detail
Purpose	Government e-way bill for moving goods worth over the threshold
Direction	Should be outbound to the NIC e-way bill system
What exists	Inbound e-way bill **numbers and files only**, typed in or uploaded when goods arrive: `lots.e_way_bill` (`models/lot.py`), `grn_imports.e_way_bill` (`models/grn_import.py`), `parts_grn.eway_bill_number / eway_bill_date / eway_bill_file` (`models/parts_grn.py` , `routers/parts_grn.py`), `crm_sourcing_deals.eway_bill_path` (`routers/crm_sourcing.py`), `e_way_bill` column in the lots CSV template (`routers/bulk_upload.py`)
What is missing	Nothing generates, validates, extends or cancels an e-way bill for outbound sales or stock transfers. `sales` has no e-way bill column (`models/sales.py`). No NIC / GSP client, credentials or API calls exist (grep for `eway` , `nic.in` , `gsp` in `.py` finds only the inbound fields above)
Status	**Not implemented**

What would be needed to build it:

1. A decision on route: direct NIC e-way bill API (needs GSTIN-level API credentials) or a GSP (GST Suvidha Provider) account.
2. Data the app does not hold today: HSN per item (only hard-coded `8471`), transporter ID / name, transport document number and date, vehicle number for outbound, distance in km, supply type and sub-type, document type, recipient GSTIN and PIN code. `sales` has `customer_state` and

`customer_address` but no customer GSTIN or PIN; `crm_contacts` has `gstin` and `pincode` but sales are not linked to a contact (`models/sales.py`, `models/crm.py`).

3. A multi-line invoice model (one invoice, many tags) so one e-way bill covers one consignment.
4. New columns or a new table for the e-way bill number, date, validity, status and cancel reason, approved on dbdiagram first (`CLAUDE.md`).
5. Secure credential storage (today secrets come from env / `config.ini`, `config.py`), retries and an audit trail (`services/audit_engine.py`).
6. Same need for internal stock transfers between entities (`stock_transfers`, `tag_number_movements`), which today only store an uploaded invoice PDF (`tag_number_movements.invoice_pdf_path`, `routers/entity_movement.py`).

2.4 GST e-invoice (IRN)

Item	Detail
Status	**Not implemented** (no IRN, IRP, signed QR or acknowledgement fields; grep finds none)
What would be needed	Registration with an IRP / GSP; a proper invoice header + line model with `Decimal` totals; HSN and UQC per line; buyer GSTIN; storage for IRN, ack number, ack date, signed QR; cancellation within the allowed window. The invoice template would need to print the QR. Same data gaps as 2.3

2.5 Gate Pass

Item	Detail
Purpose	Security gate confirms that sold tags physically left the building
Direction	Internal (user action → database)
Protocol / format	HTML page + server-side DataTables JSON; form POSTs; CSV/XLSX tag upload
Endpoints	`GET /gate-pass`, `GET /gate-pass/data`, `GET /gate-pass/ids`, `POST /gate-pass/resolve-tags` (file or text list, read-only), `POST /gate-pass/{sale_id}/exit`, `POST /gate-pass/bulk-exit` (`routers/gate_pass.py`)
Auth	Staff session + CSRF (`verify_csrf`); roles admin, sales_manager, sales, inventory_manager (`require_roles`)
Data written	`sales.delivery_status = "dispatched"`, `sales.dispatch_date` (if empty); audit `GATE_EXIT_CONFIRMED` / `GATE_EXIT_BULK_CONFIRMED`
Error handling	404 for unknown sale; already dispatched/delivered rows are skipped in bulk; bad upload → 400 "Could not read the file — upload a CSV or XLSX"
Owner	Sales / inventory roles
Status	**Live**

There is no separate gate pass number or table; the Delivery Challan (2.2) is the gate document.

2.6 Ready to Dispatch and telecaller requests

Item	Detail
Purpose	Telecaller asks to sell a Ready-to-Sale tag or a model group; manager approves, which enables the Sell button
Direction	Internal

Item	Detail
Endpoints	`GET /dispatch`, `POST /dispatch/request`, `POST /dispatch/{req_id}/approve`, `POST /dispatch/bulk-approve`, `POST /dispatch/{req_id}/reject`, `GET /inventory-requests`, `POST /inventory-requests/upload-tags` (read-only tag file match) (`routers/dispatch.py`)
Auth	Session + CSRF. View: admin, inventory_manager, sales, sales_manager, telecaller. Approve: any signed-in user, limited by permission matrix module `inventory_requests` / `edit`
Data	`telecaller_dispatch_requests` (`models/dispatch_request.py`)
Status	**Live**

2.7 Purchase Order and Quote PDFs

Item	Detail
Purpose	Downloadable PO to suppliers and quotation PDF to buyers
Format	PDF built with fpdf2; latin-1 only, so ₹ prints as "Rs " (`services/po_pdf.py`)
Endpoints	`GET /crm/sourcing/{deal_id}/download-po` (`routers/crm_sourcing.py`); quote PDF via `_quote_pdf()` in `routers/crm_quotes.py`; HTML print `GET /crm/quotes/{quote_id}/print`
Auth	Session + CSRF
Status	**Live**

2.8 Invoice PDF upload and parsing

Item	Detail
Purpose	Read invoice number, date, sender, quantity and amount from a supplier or sale invoice PDF to pre-fill forms
Direction	In (file)
Endpoints	`POST /grn/upload` (`routers/grn.py`, parser `services/invoice_parser.py` using `pypdf`); `POST /sales/parse-invoice-pdf` (`routers/sales.py`, uses `pdfplumber` if installed)
Storage	`uploads/grn/` with SHA-256 `file_hash`; `uploads/sale_invoices/inv_<epoch>_<filename>`
Error handling	Extraction is best-effort; failures leave fields blank for the user to fill (`services/invoice_parser.py` docstring)
Status	**Live, best-effort**

3. Application APIs

3.1 Session JSON API (`/api`)

Item	Detail
Purpose	Auto-fill and lookups for the web UI
Endpoints	`GET /api/lot-line-item/{item_id}`, `GET /api/lot-line-items-by-lot/{lot_id}`, `GET /api/lot-meta`, `GET /api/port-profile`, `GET /api/dealers/search`, `GET /api/device/{device_id}/next-stages` (`routers/api.py`)
Auth	Staff session cookie `access_token` (JWT HS256, `auth/dependencies.py`)

Item	Detail
Errors	404 JSON when not found
Status	**Live** (internal use only)

3.2 Machine-to-machine API (`/api/v1`)

Item	Detail
Purpose	Let other systems read inventory and register devices / move stages
Protocol	REST, JSON, Pydantic schemas in `schemas/`
Auth	`Authorization: Bearer ok_live_<64 hex>` ; only the SHA-256 hash is stored (`models/api_key.py`); scopes checked by `require_scope()` (`auth/api_key.py`). Valid scopes: `devices:read/write` , `lots:read/write` , `sales:read/write` , `iqc:read/write` , `dealers:read` , `spare_parts:read` , `intelligence:read` , `api_keys:manage`
Endpoints	`GET /api/v1/health` ; `GET /api/v1/devices` , `GET /api/v1/devices/{barcode}` , `PATCH /api/v1/devices/{barcode}/stage` ; `GET /api/v1/lots` , `/lots/{lot_number}` ; `GET /api/v1/sales` , `/sales/{sale_number}` ; `GET /api/v1/spare-parts` , `/spare-parts/{part_code}` ; `POST /api/v1/iqc/register` , `GET /api/v1/iqc/lookup` ; key admin `POST/GET/DELETE /api/v1/api-keys` (admin session); webhook admin (3.3); telecalling (5.3) (`routers/api_v1/`)
Events published	`STAGE_MOVED` (stage PATCH), `DEVICE_REGISTERED` (iqc register) (`routers/api_v1/devices.py` , `routers/api_v1/iqc.py`)
Errors	401 "Missing or malformed Authorization: Bearer header"; 401 "Invalid or revoked API key"; 403 "API key missing required scopes: ..." (`auth/api_key.py`)
CORS	Origins from `OXYPC_ALLOWED_ORIGINS` , default localhost dev ports (`config.py` , `main.py`)
Owner	Admin creates keys
Status	**Live**. No consumer of these keys is identified in the repo

3.3 Outbound webhooks and event log

Item	Detail
Purpose	Push business events to subscribers
Direction	Out
Protocol	HTTP POST, JSON, 10-second timeout (`services/webhook_dispatcher.py`)
Headers	`X-OxyPC-Signature: sha256=<hmac>` , `X-OxyPC-Event: <type>`
Signing	HMAC-SHA256 with key = the stored `secret_hash` (SHA-256 of the raw secret), not the raw secret. A receiver must hash its secret first (`_sign()`)
Event types	`DEVICE_REGISTERED` , `LOT_CREATED` , `QC_PASSED` , `SALE_COMPLETED` , `STAGE_MOVED` , `API_KEY_CREATED` , `API_KEY_REVOKED` , telecalling `CALL_LOGGED` , `CALL_ORDER_PLACED` , `CALL_QUOTE_SENT` , `CALL_DNC_SET` (`services/event_bus.py` , `services/call_service.py`)
Publishers found	`routers/sales.py` and `routers/stock.py` (`SALE_COMPLETED`), `routers/stock.py` (`LOT_CREATED`), `routers/api_v1/*` , `services/call_service.py` . No publisher of `QC_PASSED` was found
Endpoints (admin)	`POST/GET /api/v1/webhooks` , `DELETE /api/v1/webhooks/{id}` , `GET /api/v1/webhooks/{id}/events` (`routers/api_v1/webhooks.py`)
Delivery log	One `event_log` row per event: `webhook_attempts` , `last_attempt_at` , `last_status_code` (`models/event_log.py`)

Item	Detail
Error handling	Runs after the response (FastAPI background task). Failures and timeouts return `None` and are logged; there is **no retry** or dead-letter queue
Status	**Live**

4. Desktop agents and stations

4.1 OxyQC IQC machine API (`/iqc/api`)

Item	Detail
Purpose	Standalone OxyQC desktop app (PyInstaller, offline-capable per `CLAUDE.md`) registers devices and IQC results
Direction	In
Endpoints	`GET /iqc/api/health` , `GET /iqc/api/lot/{prefix}` (lot from first 4 chars of barcode), `GET /iqc/api/users` , `GET /iqc/api/check/{barcode}` , `POST /iqc/api/submit` (`routers/iqc_api.py`)
Auth	Static shared key in header `X-OxyQC-Key` , from env `OXYPC_API_KEY` or `config.ini [oxyqc] api_key` ; default value `oxyqc-default-key-change-me` (`config.py`)
Data in	`IQCSubmitPayload` : `barcode` , `lot_id` , `sub_category` , `device_type` , `brand` , `model` , `serial_no` , `grn_number` , `cpu` , `generation` , `ram_gb` , `storage_gb` , `storage_type` , `hdd_capacity_gb` , `screen_size` , `battery_health_pct` , `color` , `grade` , `floor` , `warehouse` , `notes` , `inspector_name` , `status` , `r2v3_grade_category` , IQC check fields, `charger_wattage` (stored inside a ports note, no column), `stress_report` (JSON string)
Writes	`devices` , `iqc_inspections` , `stage_movements` in one transaction
Errors	401 bad key; 404 lot not found; 409 "Barcode ... already registered"; 422 bad `lot_id`
Status	**Live**. The OxyQC desktop client source is **not in this repo** (only the server side and `tests/test_barcode_case_insensitive_creation.py` reference the endpoint)

4.2 OxyQC Diagnose agent (station hardware auto-fill)

Item	Detail
Purpose	Fill the IQC form from the hardware of the station PC, which a browser cannot read
Direction	Station agent → browser on the same PC → form fields (nothing goes to the server until the user saves)
Protocol	HTTP on loopback `127.0.0.1:8765` ; CORS `*` and `Access-Control-Allow-Private-Network: true`
Endpoints	`GET /diagnose` (or `/`), `/ping` , `/smart` , `/stress` , `/quit` (`oxyqc-agent/oxyqc_agent.py`)
Caller	`templates/iqc/form.html`
Data returned	`brand` , `model` , `serial_no` , `cpu` , `cpu_make` , `generation` , `ram_gb` , `ram_summary` , `total_ram_count/size` , `storage_gb` , `storage_type` , `hdd_capacity_gb` , `hdd_summary` , `total_hdd_count/size` , `storage_health_pct` , `sub_category` , `device_type` , `screen_size` , `battery_health_pct` , `battery_wh` , `battery_present` , `battery_cable` , port and component checks (`port_hdmi` , `port_usb_working` , `port_audio_jack` , `usb_a_ports` , `usb_c_ports` , `ethernet_ports` , `wifi_status` , `webcam_status` , `speaker_status` , `keyboard_working` , `touchpad_*` , `fan_working` , `fan_rpm` , `charging_port` , `dvd_drive` , `hdd_connector` , `hdd_casing` , `power_on` , `status` , `screen_functional`), `notes` / `_summary`

Item	Detail
Auth	None (loopback only)
Platforms	Windows (WMI/CIM via PowerShell) and macOS; agent version `2026-07-29.2`
Distribution	`GET /iqc/agent-exe` is **unauthenticated** so PXE first-boot scripts can download it (`routers/iqc.py` , `oxyqc-agent/pxe/README_PXE.txt`). Exe is unsigned (`README_AGENT.txt`)
Status	**Live**

4.3 Stress test

Item	Detail
Purpose	Run and record stress / checklist tests before cosmetic and Final QC
Endpoints	`POST /stress/{barcode}/run` , `GET /stress/{barcode}/status` , `POST /stress/{barcode}/stop` , `POST /stress/{barcode}/save` , `POST /stress/{barcode}/save-agent` (results JSON produced by the local agent on a PXE stress station, forwarded by the browser), `GET /stress/{barcode}/report` (PDF), `GET /stress/results/{barcode}` , `POST /stress/{barcode}/fail` , `POST /stress/{barcode}/complete-to-paint` , `GET /stress/has-results` (`routers/stress_api.py`)
Auth	Staff session + CSRF
Data	`stress_test_results` (`results_json` , `overall_status` , `pdf_path`); PDFs in `static/stress_reports/` ; `devices.stress_notes` on fail
Errors	422 "No results in payload"; 404 "Device not found" (only active devices)
Status	**Live**

4.4 Customer Care Agent

Item	Detail
Purpose	After-sale warranty view, support tickets, diagnostics and offers from an agent installed on the customer's PC
Direction	In / out
Public API	`/care/api/v1` : `POST /pair` , `GET /device` , `GET /warranty` , `GET /offers` , `GET/POST /tickets` , `GET /tickets/{ticket_number}` , `POST /diagnostics` , `POST /heartbeat` , `POST /token/rotate` , `POST /deactivate` (`routers/care_api.py`)
Staff APIs	`/care/internal/pairings` , `/care/internal/pairings/{id}/revoke` , `/care/internal/dispatch-readiness/{device_id}` , `/care/internal/dispatch-exceptions` (`routers/care_internal.py`); admin UI `/care-support/*` (`routers/care_admin.py`)
Auth	One-time provisioning token (30-minute TTL) exchanged at `/pair` for a device token; `Authorization: Bearer <device-token>` ; tokens stored as SHA-256 only; IPs stored hashed (`models/care.py` , `auth/care_auth.py` , `services/care_service.py`). Staff routes use session + permission module `care_support`
Format	Envelope `{success, data, request_id, server_time}` ; errors with codes such as `CARE_MISSING_TOKEN` , `CARE_INVALID_TOKEN` , `CARE_TOKEN_EXPIRED` , `CARE_DEVICE_MISMATCH` , `CARE_TICKET_NOT_FOUND`
Data in	Diagnostics allowlist only: `bios_serial` , `manufacturer` , `model` , `cpu` , `ram_gb` , `storage_summary` , `battery_health_pct` , `battery_cycle_count` , `smart_status` , `os_version` , `hardware_warning_summary` , `system_error_summary` (max 2000 chars each). Tickets: category from a fixed set, description ≤ 2000 chars, `idempotency_key` for offline retries
Rate limits	`/pair` 10/min per IP; ticket create 10/min; diagnostics 20/min; token rotate 5/min (`limiter`)

Item	Detail
Client	`care_agent_windows/` (Windows service + tray, DPAPI credential store, offline queue)
Status	**Partial.** Server side is built and registered (`main.py`). The agent README says "not a shipped product": no code signing, no installer, no update framework, no uninstall script. `care_warranties` has no writer in the code, so `/warranty` relies on the fallback in `resolve_warranty()`

5. Partner, mobile and side apps

5.1 Trade Partner portal (`/partner`)

Item	Detail
Purpose	Dealer-facing B2B catalog: browse listings and visible lots, bid, book, upload payment proof
Endpoints	`GET/POST /partner/login` , `POST /partner/logout` , `GET /partner/catalog` , `GET /partner/listings/{id}` , `POST /partner/listings/{id}/bid` , `/custom-price` , `/book` ; `GET /partner/lots/{lot_id}/models` , `POST /partner/lots/{lot_id}/bid` , `/custom-price` , `/book` , `/withdraw` ; `GET /partner/bookings` , `/bookings/{id}` , `POST /bookings/{id}/proof` , `GET /bookings/{id}/proof-file/{proof_id}` , `POST /bookings/{id}/cancel` ; `GET /partner/bids` , `GET /bids/{bid_id}/document/{doc_id}` , `POST /bids/{bid_id}/payment` ; `GET /partner/profile` ; `GET /partner/sw.js` (`routers/partner_portal.py`)
Auth	Separate JWT cookie `partner_token` , 12-hour session, own CSRF cookie `partner_csrf_token` ; login by phone + bcrypt password on `dealers.portal_phone` / `portal_password_hash` ; lockout after 5 failed attempts; `portal_password_version` invalidates old tokens (`auth/partner_auth.py` , `models/dealers.py`)
Data boundary	Must never expose `lots.buying_price` , `supplier_name` , margins or `cost_basis` (module docstring)
Data in	Bids (`partner_bids`), bookings, payment proofs (UTR, amount, screenshot), bid payments (date, mode `bank_transfer` / `upi` , UTR, amount)
File rules	Proofs: `.jpg` , `.jpeg` , `.png` , `.pdf` , max 5 MB (`services/partner_service.py`)
Staff side	`routers/partner_admin.py` (listings, Manage Lots, bids, Mark Won → creates Buyer Deal)
Status	**Live**

5.2 Partner payment verification

Item	Detail
Endpoints	`GET /partner-payments` , `GET /partner-payments/{bid_id}/details` , `POST /partner-payments/{bid_id}/verify` (`routers/partner_payments.py`)
Auth	Session + CSRF; admin, sales_manager
Data	Sets `partner_bid_payments.verified` , `verified_by` , `verified_at` , `verify_notes`
Status	**Live**. No bank or payment-gateway integration; verification is manual against the UTR

5.3 Mobile telecalling PWA

Item	Detail
Purpose	Telecallers log calls and follow-ups from a phone

Item	Detail
Endpoints	Pages under `/m/telecalling` (`routers/m_telecalling.py`); JSON under `/api/v1/telecalling` : `GET /session/today` , `GET /assigned-leads` , `POST /assignments` , `POST /calls` , `GET /calls` , `GET /calls/{call_id}/timeline` , `GET /followups/today` , `GET /kpi/dashboard` , `GET /kpi/team` , `GET /lots/{lot_id}/skus` (`routers/api_v1/telecalling.py`)
Auth	Staff session (`get_current_user`), not API keys
Data	`telecalling_records` (incl. `latitude` , `longitude` , `idempotency_key`), `telecalling_assignments` (`models/telecalling.py`)
Status	**Live**

5.4 FieldOps (Reliance asset app)

Item	Detail
Purpose	Separate field-asset app hosted in the same process; shares nothing with OxyPC data
Endpoints	`/fieldops/login` , `/fieldops/setup` , `/fieldops/api/auth/*` , `/fieldops/api/me` , `/fieldops/api/admin/users` , `/fieldops/api/admin/export` , `/fieldops/api/admin/import` , `/fieldops/api/admin/audit` , `POST /fieldops/api/sync` (offline change push, capped per push), `/fieldops/api/status` , static app `fieldops_app/` (`routers/fieldops.py`)
Auth	Own users and own session cookie (`fieldops_auth.py`); OxyPC session gives no access
Database	`FIELDOPS_DATABASE_URL` ; tables `fieldops_users` , `fieldops_records` , `fieldops_admin_audit` (`fieldops_db.py`)
Status	**Live when configured**; startup prints "FIELDOPS_DATABASE_URL not set — /fieldops disabled" otherwise (`main.py`)

6. Messaging

6.1 WhatsApp bridge (`wa-service`)

Item	Detail
Purpose (historic)	Per-user WhatsApp sessions via whatsapp-web.js: QR link, send message, groups, broadcasts
Protocol	Node service on `localhost:3001` : `GET /status` , `/qr` , `/sessions` , `/groups` , `/group-messages/:id` ; `POST /connect` , `/disconnect` , `/send` , `/send-group` , `/sync-group-messages` (`wa-service/index.js` , `start_wa.bat`)
Status	**Removed.** `main.py` comments out both the import and `app.include_router(whatsapp_router)` with "WhatsApp integration removed — router registration disabled (routes unreachable)". `routers/whatsapp.py` , `wa-service/` and tables `whatsapp_sessions` , `whatsapp_messages` , `whatsapp_groups` , `whatsapp_broadcasts` remain (`models/whatsapp.py`)
Residual	`services/care_service.py` `send_offer_whatsapp()` still POSTs to `http://localhost:3001/send` when a Care offer uses channel `whatsapp` or `both` . With the bridge gone this returns status 0 and an error, recorded as a failed `care_offer_deliveries` row
QA notes	`QA_UAT_REPORT.md` G-08 noted the module could not be tested without a real account; `docs/uat_results_2026-04-27.md` flagged the live dependency on wa-service. Both predate the removal

6.2 `wa.me` deep links

Item	Detail
Purpose	Open WhatsApp on the user's own phone with a pre-filled message
Where	`templates/trade_partner/my_desk.html`, `templates/trade_partner/bookings.html`, `templates/partner/booking_detail.html`, `templates/partner/listing_detail.html`, `templates/m/telecalling/call.html`
Auth / data	None server-side; phone and text are put in the URL
Status	**Live** (no server integration involved)

6.3 Email / SMTP

Item	Detail
Status	**Not found.** No `smtplib`, `aiosmtplib`, SendGrid or similar in application code. `users.email`, `crm_contacts.email` and `returns.customer_email` are stored but nothing sends mail. The return receipt and invoices are printed or downloaded only

---

7. Files: imports, exports, uploads

7.1 Bulk CSV upload (`/bulk-upload`)

Item	Detail
Endpoints	`GET /bulk-upload`, `GET /bulk-upload/template/{upload_type}`, `POST /bulk-upload/lots`, `/devices`, `/spare-parts`, `/leads`, plus duplicate handling `POST /bulk-upload/duplicates/approve-move`, `/update`, `/update-all`, `/add-lot` (`routers/bulk_upload.py`)
Format	CSV. Encoding detected by `utils/csv_decode.py` `decode_csv_bytes()` : UTF-16/32 only with a BOM, then UTF-8, then Windows-1252, then latin-1 (never fails)
Templates	`lots_template.csv` (lot_number, supplier_name, buying_price, qty, purchase_date, grn_*, invoice_no, invoice_value, taxable_amount, sgst, cgst, igst, po_number, vehicle_number, e_way_bill, notes); `devices_template.csv` and `devices_test_partner_template.csv` (`DEVICE_CSV_HEADERS`); `spare_parts_template.csv` (part_code, name, category, unit_price, min_stock_alert, supplier, notes); `leads_template.csv`
Auth	Session + CSRF; upload rights through `can_upload` in the permission matrix (`models/role_permissions.py`)
Validation	Stage moves go through `services/control_engine.validate_transition` ; writes are audit-logged (`services/audit_engine.py`)
Process note	Project skill `bulk-csv-import` requires asking local vs production and reporting exact skip counts (`CLAUDE.md`)
Status	**Live**

7.2 Other CSV / Excel imports

Module	Format	Source
Dealers (incl. Google Maps export, dedupe on `google_cid`)	CSV, XLSX	`routers/dealers.py`, `models/dealers.py`
Grade price matrix	CSV, XLSX	`routers/crm_price_matrix.py`
Block Tags (up to 5000 rows)	XLSX	`routers/block_tags.py`
Product Master / Pricing Calculator	XLSX	`routers/product_master.py`
Gate Pass tag list	CSV, XLSX	`routers/gate_pass.py`
Inventory Requests tag list	CSV	`routers/dispatch.py`
CRM contacts, devices, spare parts, sales	CSV	`routers/crm_contacts.py`, `routers/devices.py`, `routers/spare_parts.py`, `routers/sales.py`
FieldOps data	JSON import/export	`routers/fieldops.py`

7.3 Report exports

Item	Detail
Endpoints	`/reports/daily-stock/export/overall`, `/tags`, `/location`, `/daily-stock/export/daily-report` (XLSX, three sheets), `/reports/daily-stock/report/{date}/workbook`, `/reports/export/lot-pl`, `/reports/lot-pl/{lot_id}/export`, `/reports/business-pl/export-month`, `/reports/business-pl/export-fy`, `/reports/export/sales`, `/reports/overdue/csv` (`routers/reports.py`)
Format	CSV, and XLSX via `openpyxl` (`_daily_report_xlsx()`)
Other Excel writers	`routers/part_estimation.py`, `routers/dealers.py`, `routers/crm_price_matrix.py`, `routers/block_tags.py`, `routers/product_master.py`, `routers/gate_pass.py`
Auth	Session; export rights via `can_export` (`models/role_permissions.py`)
Status	**Live**

7.4 Daily Stock Report (scheduled)

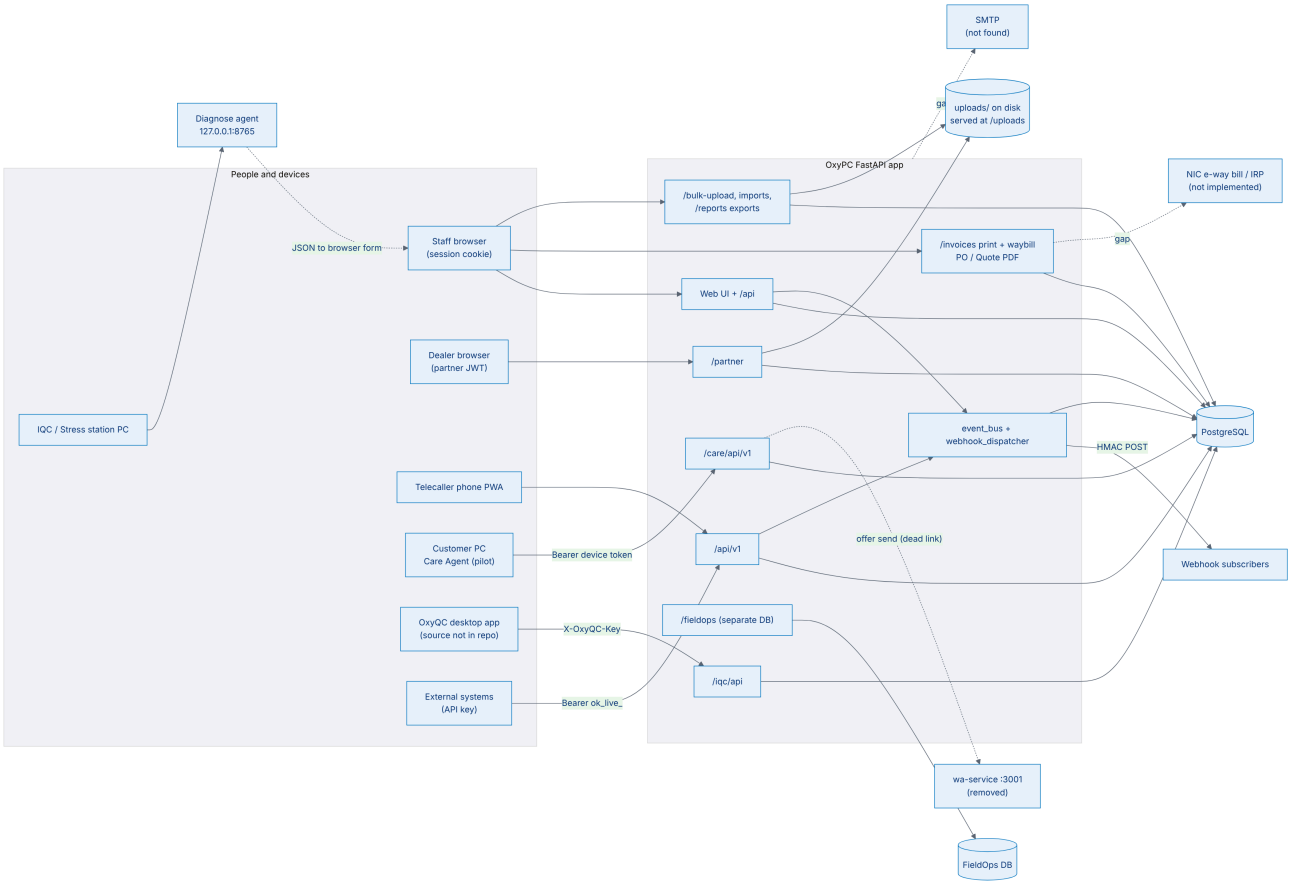
Item	Detail
Purpose	Snapshot of Overall / Tag / Location stock every day at 9:00 PM
Mechanism	Background loop started at startup (`main.py` → `services/daily_stock_reports.scheduler_loop`)
Output	`uploads/daily_stock_reports/<YYYY-MM-DD>/{overall,tags,location}.csv`; no database table
Status	**Live**

7.5 File uploads and the `/uploads` mount

Item	Detail
Storage	Local disk under `uploads/` (`config.py` `UPLOADS_DIR`): `grn/`, `sale_invoices/`, `crm/`, `partner/`, `partner/bids/`, `part_estimates/`, `entity_movement_invoices/`, `tickets/`, `daily_stock_reports/`, plus parts GRN files; stress PDFs under `static/stress_reports/`

Item	Detail
Validation	Only some routes check type/size: support ticket photos (PNG/JPEG/GIF/WEBP, 5 MB, `routers/tickets.py`) and partner proofs (JPG/PNG/PDF, 5 MB, `services/partner_service.py`). Parts GRN uploads only strip `/` and `\` from the filename (`routers/parts_grn.py` `_save_upload()`)
Serving	`app.mount("/uploads", StaticFiles(...))` in `main.py`
Security gap	The mount is plain `StaticFiles` with no auth check, so anyone who can reach the server and knows or guesses a path can download KYC documents (`crm_contacts.*_doc_path`), payment screenshots, invoices and daily stock files, bypassing the authenticated download routes (`/grn/download/{grn_id}`, `/partner/bookings/{id}/proof-file/{proof_id}`). The server is described as LAN-only (`CLAUDE.md`, `routers/iqc.py`), but `main.py` also contains Railway HTTPS-redirect handling for an internet-facing deployment
Status	**Live, needs hardening**

8. Integration map



PART D

Architecture Decision Records

These records explain the main technical decisions behind the OxyPC Inventory Lifecycle app. They were **reconstructed** from the code, the plans and specs in `docs/superpowers/`, and git history. No formal ADRs existed before this file.

How to read the history evidence: the git history in this repo starts with one large import commit, `fac334c` (22 Aug 2026, 789 files). Decisions made before that date are dated from Alembic file names (`alembic/versions/2026MMDD_…`) and sprint plans (`docs/superpowers/plans/2026-MM-DD-…`). Where a decision is older than both, the date is shown as "before 22 Aug 2026".

Status values: **Accepted** (in force), **Accepted with drift** (in force, but code and rule disagree), **Superseded**, **Deprecated**.

Index

ADR	Decision	Status	Date (best evidence)
001	Server-rendered Jinja2 pages, not a single-page app	Accepted	Before 22 Aug 2026
002	PostgreSQL with async SQLAlchemy 2 + asyncpg	Accepted	31 Mar 2026 (Alembic baseline)
003	Schema provisioned at startup by `db_validator`; Alembic secondary	Accepted with drift	Before 22 Aug 2026
004	Soft delete with `is_trashed` and a Trash page	Accepted with drift	3 May / 26 May 2026
005	Immutable audit log, written before delete	Accepted	Before 22 Aug 2026
006	In-process event bus with outbound webhooks	Accepted	30 Apr 2026
007	Custom roles and an admin-managed Permission Matrix	Accepted with drift	26 Aug 2026 onward
008	One "Global Table" component for heavy lists	Accepted	2 Sep 2026
009	Per-station localhost agent for IQC hardware diagnosis	Accepted	Before 22 Aug 2026
010	WhatsApp integration disabled	Accepted (code retained)	Before 22 Aug 2026
011	Whole app rendered at 90 % zoom	Accepted	Before 22 Aug 2026
012	Containerise the app only; one worker; poll-based auto-deploy	Accepted (supersedes Windows/bare-metal hosting)	1 Oct 2026

ADR-001 · Server-rendered Jinja2 pages, not a single-page app

Context. A small team builds many operational screens (242 templates) for floor staff on a LAN. The director gives feature requests directly (`CLAUDE.md` "People"), and the git log shows frequent page-level

changes (420 commits between 22 Aug and 21 Oct 2026). Staff use shared floor PCs and barcode scanners (`README.md` "Barcode Scanner").

Decision. Every page is rendered on the server by FastAPI routes with Jinja2 templates, styled with Bootstrap 5 and enhanced with jQuery, DataTables and small vanilla JS files. All libraries are stored in `static/vendor/` (no CDN). When other systems needed data, a separate JSON API (`/api/v1/`) was added **alongside** the HTML routes rather than converting the UI ("Parallel routes — all existing HTML routes remain unchanged", `docs/superpowers/plans/2026-04-29-sprint17a-ecosystem-foundation.md`).

Consequences. - One codebase and one deploy; permissions are enforced in the same place pages are built (`templates_config.py` globals such as `has_perm` , `role_allowed`). - Fast to add a page; no build step for the front end. - Interactivity needs extra patterns: AJAX partial refresh with `DOMParser` swaps (`.claude/skills/ajax-partial-refresh`), form autosave (`static/js/form-autosave.js`). - Big tables can produce multi-MB HTML (`deploy/nginx-oxypc-internal.conf` comment), which led to ADR-008. - Pages are sent `Cache-Control: no-store` and assets are cache-busted so deploys show at once (`main.py` `_no_store_pages` , `templates_config.py` `ASSET_VERSION`).

Source. `main.py` , `templates_config.py` , `templates/base.html` , `static/vendor/` , sprint 17a plan, design system checklist "No CDN links".

ADR-002 · PostgreSQL with async SQLAlchemy

Context. 30+ users write at the same time (stage moves, IQC entries, sales). Data is relational (lots → devices → stage movements → sales) and needs transactions, enums and UUID keys.

Decision. Use PostgreSQL with SQLAlchemy 2.0 in async mode and the asyncpg driver. One `AsyncSession` per request, committed after the handler and rolled back on error. Standard query form: `await db.execute(select(...))` then `.scalar_one_or_none()` / `.scalars().all()` .

Consequences. - Good concurrency on a single async worker (ADR-012). - Every handler must `await` DB calls; blocking calls (for example `subprocess.run` in `POST /admin/backup-now` , `routers/admin.py`) stall all users. - Pool size must stay under PostgreSQL `max_connections` (`database.py` comment: 8 + 4 per process). - Raw SQL is still used for reports and migrations (`text(...)` in `db_validator.py` , `migrate_*.py`).

Source. `database.py` , `requirements.txt` (`sqlalchemy[asyncio]==2.0.36` , `asyncpg==0.30.0`), `CLAUDE.md` "Key pattern", `alembic/versions/20260331_1600_ba22dd09a6a3_initial_baseline.py` .

ADR-003 · Schema is provisioned at startup by `db_validator` ; Alembic is secondary

Context. New columns ship often. Missing columns in production caused "silent 500s" (`db_validator.py` header). Alembic revisions were not always applied to production.

Decision. On every start, `db_validator.validate_and_fix()` compares ORM models to the database, creates missing tables, adds missing columns (`ADD COLUMN IF NOT EXISTS`), reseeds stages and transitions, and aborts startup if something cannot be fixed. Alembic (36 revisions) and one-off `migrate_*.py` scripts remain for indexes, type changes and data moves. New tables or columns need dbdiagram.io approval first.

Consequences. - Deploys "just work" for additive changes; the Docker auto-deploy needs no migration step (`deploy/oxypc-autodeploy.sh` comment). - Non-additive changes (rename, drop, type change, index) are not covered and need manual scripts. - Alembic history no longer describes production: `20260925_1200_add_dashboard_perf_indexes.py` found indexes from an April revision "never actually present on the live schema". - Three overlapping mechanisms (validator, Alembic, 51 `migrate_*.py`) make the true schema hard to know.

Source. `db_validator.py`, `main.py` startup, `upgrade_db.py` header, `.github/workflows/deploy.yml` comment ("evolves the schema via the additive-column auto-provisioner … NOT hand-written Alembic migrations"), `CLAUDE.md` Critical Rules.

ADR-004 · Soft delete with `is_trashed` and a Trash page

Context. Devices, lots and parts carry financial and audit history. Hard deletes broke reports and made mistakes unrecoverable.

Decision. Mark rows with `is_trashed = true` instead of deleting them; filter them out of lists; restore from a Trash page. Devices also have `is_active`. The single remaining `db.delete(device)` was replaced (sprint 21).

Consequences. - Mistakes can be undone; history stays intact. - Every query must remember the filter (187 `is_trashed` references in routers/services/models). - Rule drift: `CLAUDE.md` says the column is nullable and must be filtered with `isnot(True)`. The models now declare it `nullable=False` with a `false` default (`models/device.py` l.248, `models/lot.py` l.47), and most code uses `== False` (126 vs 10). Both are correct today; the rule text should be updated. - 36 `db.delete()` calls still exist for other records, so ADR-005 matters.

Source. `alembic/versions/20260503_0900_soft_delete_devices.py`, `alembic/versions/20260526_1400_add_trash_to_lots_devices.py`, `docs/superpowers/plans/2026-05-08-sprint21-pagination-softdelete.md`, `routers/trash.py`, commit `7e72210` ("Trash duplicates").

ADR-005 · Immutable audit log, written before delete

Context. The director needs to know who changed what (stage moves, prices, deletions). After `db.delete()` and commit, the record's values are gone.

Decision. A single helper, `services/audit_engine.audit()`, writes an `AuditLog` row (user, IP, action, table, record id, old and new values, notes). It does not commit; it joins the caller's transaction, so the audit row and the change succeed or fail together. Rule: call it **before** `db.delete()`. The table is documented as immutable ("No UPDATE or DELETE permitted").

Consequences. - 242 call sites; a full audit-log page for admins (`routers/admin.py`). - Immutability is by convention, not enforced by database permissions or triggers. - No retention policy; the table will grow without limit. - Some startup recoveries rely on it (for example `recover_from_audit_once` in `services/grn_stocked.py`).

Source. `services/audit_engine.py`, `models/engines.py` (`AuditLog`), `CLAUDE.md` Critical Rules, `alembic/versions/20261001_1000_audit_records_my_fields.py`.

ADR-006 · In-process event bus with outbound webhooks

Context. Other "ecosystem" apps (Customer Portal, AI layer, ESG, Finance) need to hear about key events in real time (`config.py` CORS comment; sprint 17a/17b plans).

Decision. A lightweight in-process pub/sub (`services/event_bus.py`). Routes call `publish()` after commit; handlers run as FastAPI `BackgroundTasks` after the response. At startup the webhook dispatcher subscribes to 7 event types and sends HMAC-signed HTTP calls to registered webhooks; deliveries are logged in `event_log` .

Consequences. - No message broker to run; simple to operate. - Events are lost if the process dies between response and delivery; there is no retry queue in the bus itself. - Only 10 `publish(EventType…)` calls exist, so coverage of business events is partial. - Works only within one process; more workers would each run their own handlers (fine for webhooks, but noted).

Source. `docs/superpowers/plans/2026-04-30-sprint17b-event-system.md` , `services/event_bus.py` , `services/webhook_dispatcher.py` , `main.py` startup (subscribe loop), `alembic/versions/20260430_0900_add_webhooks_event_log.py` .

ADR-007 · Custom roles and an admin-managed Permission Matrix

Context. The original fixed roles (admin, inventory_manager, sales, iqc_inspector, l1/l2/l3_engineer, qc_inspector — `CLAUDE.md`) did not match how teams actually split work (e.g. `trc_manager` , cosmetic leads, External Partner pages). Every change needed a code edit.

Decision. Admins create custom roles and tick modules per role in Master Data → Module Permissions. The sidebar and the backend both read this matrix (cached, refreshed every 60 s). `require_roles()` lets custom roles through non-admin gates and lets the matrix grant admin-only modules explicitly. In templates, `role_allowed()` mirrors this so buttons match the backend. A `sub_admin` role is seeded at startup with no permissions.

Consequences. - New roles need no deploy. - Two layers (built-in allow-lists + matrix) make "why is this 403?" harder to answer; `_SEGMENT_MODULE_ALIASES` exists to close gaps where the nav and routes disagree. - Drift: 15 template checks still hard-code `current_user.role.value` , so custom roles can miss buttons. The `custom-role-permissions` skill exists to stop new cases.

Source. `auth/dependencies.py` (`require_roles` , `_matrix_grants_path` , `require_module_perm`), `templates_config.py` (`_role_allowed`), `models/role_permissions.py` , `main.py` (sub_admin seed, cache refresh), `.claude/skills/custom-role-permissions` , commits `5a0b33e` (real Edit/Add permissions), `d862fa7` (External Partner split into 8 pages).

ADR-008 · One "Global Table" component for heavy lists

Context. Inventory Search holds "~19-23k rows — the heaviest table in the app" (commit `ae43e3f`). Each page had its own DataTables setup, so headers wrapped, columns squashed and behaviour varied.

Decision. One module, `static/js/global-table.js` , applied to server-side DataTables: single-line headers, horizontal scroll with a frozen first column (second too if the first is a checkbox) using CSS

`position: sticky`, 12 rows by default, 32-character clamp with tooltip, a standard toolbar (page view, row count, search/scan, pagination) and optional scan-to-select. It is part of the cache-busting asset list.

Consequences. - Consistent tables across 49 templates that reference it; 13 use full server-side paging. - One file is now critical: a change affects every big list, so `regression-sweep` is required when shared UI changes. - A cross-browser bug (2026-09-03) came from caching, not rendering, and was fixed by adding the file to `_VERSIONED_ASSETS` (`tests/test_asset_version_cache_busting.py`).

Source. `static/js/global-table.js` header, commits `ae43e3f`, `097c736`, `108a742`, `5583f89`, `8aa8032`, `templates_config.py`.

ADR-009 · Per-station localhost agent for IQC hardware diagnosis

Context. IQC inspectors type CPU, RAM, storage and serial details for every device. "A browser cannot read the hardware of the PC it runs on" (`oxyqc-agent/README_AGENT.txt`). Earlier, a standalone OxyQC EXE submitted inspections through a machine API.

Decision. Each inspection station runs a small agent (`Diagnose_Device_Agent.exe`, from `oxyqc-agent/oxyqc_agent.py`) that listens only on `127.0.0.1:8765`. The IQC page's JavaScript calls it to auto-fill hardware fields. It self-installs per user with no admin rights. The separate OxyQC app keeps using `/iqc/api/*` with an `X-OxyQC-Key` header. A GitHub Action rebuilds the agent EXE when its source changes. PXE-deployed station images (IQC line and Stress Test / Final QC) download the latest agent from `<SERVER_URL>/iqc/agent-exe` on each boot (`oxyqc-agent/pxe/README_PXE.txt`).

Consequences. - No inbound firewall rules; the agent is never exposed on the LAN. - One more piece of software to install and update on every station; the EXE is committed to git. - Hardware quirks need agent fixes (commit `8835f10`: agent "dies silently on some hardware's blank SMBIOS data"). - The OxyQC API key has an insecure default in `config.py` if not set.

Source. `oxyqc-agent/README_AGENT.txt`, `oxyqc-agent/`, `routers/iqc_api.py`, `config.py` (`OXYQC_API_KEY`), `.github/workflows/build-agent.yml`, commit `8835f10`.

ADR-010 · WhatsApp integration disabled

Context. A WhatsApp integration existed: a Node service (`wa-service/`), a router and model (`routers/whatsapp.py`, `models/whatsapp.py`), templates (`templates/whatsapp/`) and a start script (`start_wa.bat`).

Decision. Disable it by commenting out the router import and registration, so its routes are unreachable. The code stays in the repo.

Consequences. - No WhatsApp messaging from the ERP; account-level WhatsApp CSV columns were also dropped from CRM bulk upload (commit `12388b1`). Manuals omit WhatsApp pages (commit `0c496bf`). - Dead code remains (router, model, ~1.7 GB local `wa-service/` install excluded via `.dockerignore`, docker spec §9). - The Care Agent plan still mentions WhatsApp offers (`docs/care-agent/phase5-offers-whatsapp.md`), so the topic may return. - The reason for disabling is not recorded in code or commits.

Source. `main.py` lines 176 and 266-267 ("WhatsApp integration removed — router registration disabled (routes unreachable)"), present since `fac334c`; commits `12388b1`, `0c496bf`.

ADR-011 · Whole app rendered at 90 % zoom

Context. Pages show wide tables and dense forms. The reason for the global zoom is not written down; the most likely aim is to fit more on screen without horizontal scrolling (inferred, see Open questions).

Decision. Scale the whole UI with `body { zoom: var(--app-zoom) }`, `--app-zoom: 0.9`. Anything sized with `vh` must use `--full-vh` (`100vh / 0.9`). The design system documents all sizes "before that zoom".

Consequences. - More data fits on screen on every PC with no per-user setup. - Side effects need fixes: modal backdrop sized with `100%` instead of viewport units (`static/css/app.css` l.23-30); Chart.js hover misaligned until `static/js/chart-zoom-fix.js` (commit `eb8b4e2`); fixed-position dropdowns rendered ~10 % off because Chromium applies the zoom twice to `position: fixed` elements (commit `8ef7df4`). - Effective table text is about 12.6 px, which may hurt readability (see NFR-13, NFR-15). - Depends on CSS `zoom`, which is non-standard in older Firefox versions.

Source. `static/css/app.css` l.10-20, `qa_docs/design_system/design_system.html` (typography and tokens), commits `eb8b4e2`, `8ef7df4`.

ADR-012 · Containerise the app only; one worker; poll-based auto-deploy

Context. The app first ran on a Windows laptop (`192.168.1.100:8000`, scheduled tasks, NSSM, watchdog) and then on a Linux Internal Server as a bare-metal venv with systemd. The bare-metal setup had "no environment reproducibility", "no rollback primitive" and "no blast-radius isolation" (docker spec §1). The server has a private IP that GitHub cannot reach.

Decision. 1. Run the app in one Docker container (`python:3.12-slim`, non-root uid 1000) with host networking; keep PostgreSQL native on the host for now ("the one component where a mistake is genuinely hard to undo", spec §3). 2. Run **one** uvicorn worker, because permission/transition caches and the rate limiter live in process memory. 3. Deploy by a systemd timer that polls `origin/main` every 2 minutes, refuses to deploy over local edits, rebuilds the image, polls `/health`, and does **not** auto-roll back. 4. Keep the disabled `oxypc.service` as the rollback path.

Consequences. - Cutover took ~8 s of downtime; rollback is "run the previous commit/image". - Uploads/backups are bind mounts, so data survives rebuilds; they needed a one-time `chown` to uid 1000. - Files written inside the container but not mounted (e.g. `errors.log`) are lost on each deploy. - Single worker means any blocking call freezes all users; scaling out needs a shared cache/limiter first. `deploy/oxypc.service` still claims 4 workers are safe, which conflicts with the `Dockerfile`. - `.github/workflows/deploy.yml` (self-hosted, `systemctl restart oxypc`) predates this decision and may now be obsolete. - Postgres containerisation is deferred as "Phase 2".

Supersedes. Windows hosting scripts (`run_server.ps1`, `setup_autostart.ps1`, `watchdog_server.ps1`, `install_service.bat`) and bare-metal `deploy/oxypc.service` for production.

Source. `docs/superpowers/specs/2026-10-01-docker-containerization-design.md`, `Dockerfile`, `docker-compose.yml`, `deploy/oxypc-autodeploy.sh`, `deploy/oxypc-autodeploy.timer`, commits `992255e`,

`efc7388`, `e3a90c9`, `8feadf0`.

Open questions

1. Why was the WhatsApp integration disabled (cost, policy, reliability)? Should the code be deleted or revived for the Care Agent offers?
2. Who decided on the 90 % zoom, and was it tested on the smallest floor monitors?
3. Should Alembic become the only migration path (ADR-003), or should the validator stay the main mechanism?
4. Is moving to multiple workers (ADR-012) planned? It needs a shared rate-limit store and a single owner for scheduled jobs.
5. Should audit-log immutability (ADR-005) be enforced in the database (revoke UPDATE/DELETE)?
6. Is the standalone OxyQC app still used, or has the browser IQC page with the localhost agent fully replaced it (ADR-009)?
7. Git history before 22 Aug 2026 is not in this repository. Is there an older repository (for example `TheCodeOrbit/oxypc`, named in `installer.py`) with the original decision history?